

Guía Práctica de WPF y XAML

*Elementos, conceptos, propiedades y
atributos con ejemplos reales paso a paso*



Eduardo Vargas Solanas

Profesor del módulo Desarrollo de Interfaces

CIP FP Distancia de Logroño

Índice

Introducción.....	2	3.10 <ListView>	35
1. Contenedores de diseño.....	2	3.11 <DataGrid>.....	36
1.1 <Grid>	2	3.12 <Calendar>.....	37
1.2 <StackPanel>	3	3.13 <DatePicker>.....	38
1.3 <WrapPanel>.....	4	3.14 <Frame>	39
1.4 <DockPanel>.....	6	Ejemplo en code-behind Hay que crear Page1 y Page2, explicado más adelante .	40
1.5 <Canvas>	7	3.15 <Page>	41
1.6 <UniformGrid>	8	3.16 <NavigationWindow>	42
2. Controles Comunes	9	3.17 <Hyperlink>.....	43
2.1 <TextBox>	10	4. Estilos, Gráficos y Efectos.....	44
2.2 <Label>	11	4.1 <Ellipse>	44
2.3 <CheckBox>	12	4.2 <Rectangle>.....	45
2.4 <RadioButton>	14	4.3 <Line>	47
2.5 <ComboBox>	15	4.4 <Path>.....	48
2.6 <ListBox>	16	4.5 <Transformaciones> (RotateTransform, ScaleTransform, TranslateTransform, SkewTransform, TransformGroup).....	49
2.7 <Slider>	18	4.6 <Border>	50
2.9 <ProgressBar>	19	4.7 <Style>.....	51
2.10 <PasswordBox>	20	4.8 <Trigger>.....	53
2.11 <Image>	22	4.9 <EventTrigger>	54
2.12 <TextBlock>.....	23	4.10 <MultiTrigger>	55
3. Controles Avanzados	24	4.11 <DataTrigger>	56
3.1 <TabControl y TabItem>	24	4.12 <Storyboard>	58
3.2 <GroupBox>	26	4.13 <VisualBrush>	59
3.3 <Expander>.....	27	4.14 <DrawingBrush>	60
3.4 <ScrollViewer>	29	4.15 <ImageBrush>	61
3.6 <Menu> y <MenuItem>.....	30	4.16 <DropShadowEffect>.....	62
3.7 <ToolBar>	31	4.17 <BlurEffect>	63
3.8 <StatusBar>	32	5. CONCEPTOS ADICIONALES BREVES.....	64
3.9 <TreeView>	34	5.1 <x:Name>	64

5.2 <Visibility>	65	5.7 StaticResource.....	74
5.3 <ItemsSource>	67	5.8 DynamicResource	75
5.4 SelectedItem	69	5.9 ElementName	76
5.5 SelectedIndex	71	5.10 ResourceDictionary	77
5.6 DataContext	72		

Introducción

Cuando Visual Studio crea un UserControl, por defecto pone un `<Grid></Grid>` vacío dentro. El Grid es excelente si vas a hacer un formulario complejo con filas y columnas (como una tabla de Excel).



La regla de oro en WPF es: usa el contenedor más simple que haga el trabajo.

1. Contenedores de diseño

1.1 `<Grid>`

¿Qué es `<Grid>`?

Contenedor que organiza elementos en filas y columnas. Es el sistema de diseño más poderoso y flexible en WPF.

¿Para qué se usa?

Se utiliza para distribuir el espacio de manera estructurada, adaptable y precisa en interfaces de usuario complejas o dinámicas.

¿Dónde es mejor usarlo?

Cuando necesitas diseñar interfaces que requieran una alineación precisa de múltiples elementos en dos dimensiones.

¿Cuándo es mejor evitarlo?

En interfaces extremadamente simples donde un StackPanel u otro contenedor más ligero sería suficiente.

Ejemplo básico

```
<Grid>
  <Grid.RowDefinitions>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="*/>
  </Grid.RowDefinitions>
  <Button Grid.Row="0" Content="Aceptar"/>
```

```
<Button Grid.Row="1" Content="Cancelar"/>
</Grid>
```

Ejemplo avanzado con filas, columnas y propiedades adicionales

```
<Grid ShowGridLines="True">
  <Grid.RowDefinitions>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="*" />
  </Grid.RowDefinitions>
  <Grid.ColumnDefinitions>
    <ColumnDefinition Width="2*" />
    <ColumnDefinition Width="*" />
  </Grid.ColumnDefinitions>

  <Button Grid.Row="0" Grid.Column="0" Content="Aceptar" Margin="5"/>
  <Button Grid.Row="0" Grid.Column="1" Content="Cancelar" Margin="5"/>
  <TextBlock Grid.Row="1" Grid.ColumnSpan="2" Text="Mensaje final"
    HorizontalAlignment="Center" Margin="10"/>
</Grid>
```

Atributos comunes de <Grid>

- RowDefinitions: Define las filas.
- ColumnDefinitions: Define las columnas.
- Grid.Row: Ubica el control en una fila específica.
- Grid.Column: Ubica el control en una columna específica.
- Grid.RowSpan: Hace que un control ocupe varias filas.
- Grid.ColumnSpan: Hace que un control ocupe varias columnas.
- ShowGridLines: Muestra las líneas de la cuadrícula (útil para depuración).
- Margin: Define el espacio exterior alrededor del control.
- HorizontalAlignment y VerticalAlignment: Alinean el contenido dentro de su celda.

1.2 <StackPanel>

¿Qué es <StackPanel>?

Es un contenedor que organiza los elementos hijos en una sola línea, ya sea horizontal o verticalmente.

¿Para qué se usa?

Se utiliza para crear listas de controles o formularios simples donde los elementos deben apilarse en orden.

¿Dónde es mejor usarlo?

En interfaces sencillas donde la distribución lineal de los elementos es suficiente y no se necesita alineación compleja.

¿Cuándo es mejor evitarlo?

Cuando se requiere un control más preciso de la alineación cruzada o distribución proporcional entre elementos (mejor usar Grid).

Ejemplo básico

```
<StackPanel Orientation="Vertical">
  <Button Content="Botón 1"/>
  <Button Content="Botón 2"/>
</StackPanel>
```

Ejemplo avanzado con alineación y márgenes

```
<StackPanel Orientation="Horizontal" HorizontalAlignment="Center" Margin="10">
  <Button Content="Aceptar" Margin="5"/>
  <Button Content="Cancelar" Margin="5"/>
</StackPanel>
```

Atributos comunes de <StackPanel>

- Orientation: Define si los elementos se apilan Vertical u Horizontalmente.
- Margin: Define el espacio exterior del StackPanel.
- HorizontalAlignment y VerticalAlignment: Alinean el StackPanel dentro de su contenedor padre.
- Background: Define el color de fondo.
- Padding: Define el espacio interno alrededor del contenido del StackPanel.

1.3 <WrapPanel>

¿Qué es <WrapPanel>?

Es un contenedor que organiza los elementos hijos en una línea y los envuelve automáticamente a la siguiente cuando no caben en el espacio disponible.

¿Para qué se usa?

Se utiliza para diseñar interfaces dinámicas que requieren adaptabilidad en función del tamaño disponible, como galerías de imágenes, etiquetas o iconos.

¿Dónde es mejor usarlo?

En interfaces donde los elementos deben fluir y adaptarse automáticamente al tamaño del contenedor sin necesidad de un diseño fijo.

¿Cuándo es mejor evitarlo?

Cuando se requiere un control estricto sobre la alineación precisa o distribución de los elementos, ya que WrapPanel prioriza el flujo sobre el control exacto.

Ejemplo básico

```
<WrapPanel>
  <Button Content="1"/>
  <Button Content="2"/>
  <Button Content="3"/>
  <Button Content="4"/>
</WrapPanel>
```

Ejemplo avanzado con orientación y tamaño de ítems

```
<WrapPanel Orientation="Horizontal" Margin="10">
  <Button Content="Botón A" Width="100" Height="50" Margin="5"/> <Button
Content="Botón B" Width="100" Height="50" Margin="5"/>
  <Button Content="Botón C" Width="100" Height="50" Margin="5"/>
  <Button Content="Botón D" Width="100" Height="50" Margin="5"/>
</WrapPanel>
```

Atributos comunes de <WrapPanel>

- Orientation: Define si los elementos fluyen Horizontal o Verticalmente.
- ItemWidth: Define el ancho fijo de cada elemento dentro del WrapPanel.
- ItemHeight: Define la altura fija de cada elemento.

- Margin: Define el espacio exterior del WrapPanel.
- HorizontalAlignment y VerticalAlignment: Alinean el WrapPanel dentro de su contenedor padre.
- Background: Define el color de fondo del WrapPanel.

1.4 <DockPanel>

¿Qué es <DockPanel>?

Es un contenedor que alinea los elementos hijos a uno de sus lados (Top, Bottom, Left, Right) permitiendo un diseño clásico de aplicación de escritorio.

¿Para qué se usa?

Se utiliza para crear interfaces donde los elementos deben estar anclados a los bordes del contenedor, como menús, barras de herramientas o paneles laterales.

¿Dónde es mejor usarlo?

En aplicaciones donde se requiere separar claramente zonas funcionales como encabezados, menús laterales o pies de página.

¿Cuándo es mejor evitarlo?

Cuando se necesita un diseño adaptable más flexible o centrado, en cuyo caso Grid puede ser más adecuado.

Ejemplo básico

```
<DockPanel>
  <Button Content="Encabezado" DockPanel.Dock="Top"/>
  <Button Content="Menú Izquierda" DockPanel.Dock="Left"/>
  <TextBlock Text="Contenido Principal"/>
</DockPanel>
```

Ejemplo avanzado con último elemento llenando espacio

```
<DockPanel LastChildFill="True">
  <Button Content="Top" DockPanel.Dock="Top"/> <Button Content="Left"
  DockPanel.Dock="Left"/>
  <Button Content="Right" DockPanel.Dock="Right"/>
```



```
<TextBlock Text="Área central llena" Background="LightGray"/>
</DockPanel>
```

Atributos comunes de <DockPanel>

- DockPanel.Dock: Define a qué lado del DockPanel se ancla un elemento (Top, Bottom, Left, Right).
- LastChildFill: Indica si el último hijo debe rellenar el espacio restante (por defecto es True).
- Margin: Define el espacio exterior de los elementos.
- HorizontalAlignment y VerticalAlignment: Alinean los elementos dentro del DockPanel.
- Background: Define el color de fondo del DockPanel.

1.5 <Canvas>

¿Qué es <Canvas>?

Es un contenedor que permite posicionar elementos de forma absoluta mediante coordenadas X e Y.

¿Para qué se usa?

Se utiliza para gráficos, juegos, diseños libres o situaciones donde se necesita un control total sobre la ubicación de los elementos.

¿Dónde es mejor usarlo?

En escenarios donde se necesita posicionar manualmente cada control o en interfaces altamente personalizadas como editores gráficos.

¿Cuándo es mejor evitarlo?

Cuando se requiere un diseño responsivo o adaptable a distintos tamaños de pantalla.

Ejemplo básico

```
<Canvas>
  <Button Canvas.Left="50" Canvas.Top="30" Content="Botón 1"/>
  <Button Canvas.Left="150" Canvas.Top="100" Content="Botón 2"/>
</Canvas>
```

Ejemplo avanzado con alineación y fondo

```
<Canvas Background="LightGray" Width="400" Height="300">  
  <Rectangle Canvas.Left="20" Canvas.Top="20" Width="100" Height="50" Fill="Blue"/>  
  <Ellipse Canvas.Left="150" Canvas.Top="80" Width="80" Height="80" Fill="Red"/>  
  <TextBlock Canvas.Left="100" Canvas.Top="200" Text="Texto flotante" FontSize="16"/>  
</Canvas>
```

Atributos comunes de <Canvas>

- Canvas.Left: Define la distancia del elemento desde el borde izquierdo del Canvas.
- Canvas.Top: Define la distancia del elemento desde el borde superior del Canvas.
- Canvas.Right: Define la distancia del elemento desde el borde derecho (menos común).
- Canvas.Bottom: Define la distancia del elemento desde el borde inferior (menos común).
- Background: Define el color de fondo del Canvas.
- Width y Height: Dimensiones explícitas del Canvas.
- Margin: Espaciado exterior alrededor del Canvas.

1.6 <UniformGrid>

¿Qué es <UniformGrid>?

Es un contenedor que organiza automáticamente los elementos hijos en una cuadrícula donde todas las celdas tienen el mismo tamaño.

¿Para qué se usa?

Se utiliza para crear diseños simétricos como teclados, tableros de juegos o menús de opciones donde cada elemento ocupa el mismo espacio.

¿Dónde es mejor usarlo?

En situaciones donde se requiere un diseño limpio y uniforme de los elementos, sin necesidad de configurar filas y columnas manualmente.

¿Cuándo es mejor evitarlo?

Cuando necesitas control individual sobre el tamaño de filas o columnas, en cuyo caso Grid sería más apropiado.

Ejemplo básico

```
<UniformGrid Rows="2" Columns="2">  
  <Button Content="A"/> <Button Content="B"/>  
  <Button Content="C"/>  
  <Button Content="D"/>  
</UniformGrid>
```

Ejemplo avanzado con más elementos

```
<UniformGrid Rows="4" Columns="3" Margin="10" HorizontalAlignment="Center"  
VerticalAlignment="Center">  
  <Button Content="1" Click="BotonNumerico_Click"/>  
  <Button Content="2" Click="BotonNumerico_Click"/>  
  <Button Content="3" Click="BotonNumerico_Click"/>  
  <Button Content="4" Click="BotonNumerico_Click"/>  
  <Button Content="5" Click="BotonNumerico_Click"/>  
  <Button Content="6" Click="BotonNumerico_Click"/>  
  <Button Content="7" Click="BotonNumerico_Click"/>  
  <Button Content="8" Click="BotonNumerico_Click"/>  
  <Button Content="9" Click="BotonNumerico_Click"/>  
  <Button Content="C" Click="BotonLimpiar_Click" Background="LightCoral"/>  
  <Button Content="0" Click="BotonNumerico_Click"/>  
  <Button Content="OK" Click="BotonAceptar_Click" Background="LightGreen"/>  
</UniformGrid>
```

Atributos comunes de <UniformGrid>

- Rows: Define el número de filas en la cuadrícula.
- Columns: Define el número de columnas en la cuadrícula.
- FirstColumn: Define en qué columna debe empezar el primer elemento.
- Margin: Espaciado exterior alrededor del UniformGrid.
- HorizontalAlignment y VerticalAlignment: Alinean el UniformGrid dentro de su contenedor padre.
- Background: Color de fondo del UniformGrid.

2. Controles Comunes

Esta guía explica el uso de los controles comunes en WPF de forma completa y profesional. Cada control incluye una descripción, usos recomendados, ejemplos básicos y avanzados, y atributos comunes.

2.1 <TextBox>

¿Qué es?

Control de entrada de texto que permite al usuario escribir datos.

¿Para qué se usa?

Capturar texto de usuario, como nombres, correos, comentarios o búsquedas.

¿Dónde es mejor usarlo?

Formularios de registro, filtros de búsqueda, comentarios de usuarios.

¿Cuándo evitarlo?

Evitar si el contenido es solo de visualización (usar TextBlock).

Ejemplo básico en XAML

```
<TextBox
    Width="250"
    Height="40"
    Background="LightYellow"
    Foreground="DarkBlue"
    FontSize="14"
    FontWeight="Bold"
    FontFamily="Segoe UI"
    TextAlignment="Center"
    Margin="10"
    Padding="5"
    Text="Introduce tu nombre aquí"/>
```

Ejemplo avanzado en XAML

```
<StackPanel Margin="20">
    <Label Content="Nombre:"/>
    <TextBox Name="txtNombre" Width="200" Height="30" Margin="0,5"/>
    <Button Content="Enviar" Width="100" Margin="0,10" Click="ButtonEnviar_Click"/>
</StackPanel>
```

Ejemplo en code-behind

```
private void ButtonEnviar_Click(object sender, RoutedEventArgs e)
{
    MessageBox.Show($"Hola {txtNombre.Text}, bienvenido.");
}
```

Atributos comunes

- Text: Contenido del TextBox.
- Width, Height: Tamaño del control.
- FontSize, FontWeight, FontFamily: Estilos de fuente.
- TextAlignment: Alineación del texto.
- Background, Foreground: Colores de fondo y texto.
- Margin, Padding: Espaciados exteriores e interiores.

2.2 <Label>

¿Qué es?

Control que muestra texto descriptivo no editable.

¿Para qué se usa?

Mostrar etiquetas informativas junto a campos o controles.

¿Dónde es mejor usarlo?

Formularios, guías de usuario, identificadores de campos.

¿Cuándo evitarlo?

Evitar si se necesita interacción o edición (usar TextBox).

Ejemplo básico en XAML

```
<Label
    Content="Nombre:"
    FontSize="14"
    FontWeight="SemiBold"
    FontFamily="Arial"
    Foreground="DarkGreen"
    HorizontalAlignment="Left"
    Margin="10"/>
```

Ejemplo avanzado en XAML

```
<StackPanel Margin="20">
    <Label Content="Introduce tu usuario:" FontWeight="Bold"/>
    <TextBox Name="txtUsuario" Width="200" Margin="0,5"/>
    <Button Content="Validar" Width="100" Margin="0,10" Click="ButtonValidar_Click"/>
    <Label Name="lblError" Content="" Foreground="Red" Visibility="Collapsed"/>
</StackPanel>
```

Ejemplo en code-behind

```
private void ButtonValidar_Click(object sender, RoutedEventArgs e)
{
    lblError.Visibility = string.IsNullOrEmpty(txtUsuario.Text) ? Visibility.Visible :
    Visibility.Collapsed;
    lblError.Content = "El campo usuario es obligatorio.";
}
```

Atributos comunes

- Content: Texto o contenido visual.
- FontSize, FontWeight, FontFamily: Estilos de texto.
- Foreground: Color del texto.
- HorizontalAlignment, VerticalAlignment: Alineaciones.
- Margin: Espaciado exterior.

2.3 <CheckBox>

¿Qué es?

Control que permite seleccionar o deseleccionar una opción.

¿Para qué se usa?

Permitir al usuario aceptar condiciones o activar configuraciones.

¿Dónde es mejor usarlo?

Formularios de aceptación, ajustes de usuario.

¿Cuándo evitarlo?

Evitar si solo una opción debe seleccionarse (usar RadioButton).

Ejemplo básico en XAML

```
<CheckBox
    Content="Acepto los términos"
    Width="200"
    Height="30"
    FontSize="13"
    FontWeight="Normal"
    Margin="10"
    Foreground="Black"/>
```

Ejemplo avanzado en XAML

```
<StackPanel Margin="20">
    <CheckBox Name="chkCondiciones" Content="Acepto las condiciones" Margin="0,5"
        Checked="CheckBox_Checked" Unchecked="CheckBox_Unchecked"/>
    <Button Name="btnContinuar" Content="Continuar" IsEnabled="False" Width="100"
        Margin="0,10"/>
</StackPanel>
```

Ejemplo en code-behind

```
private void CheckBox_Checked(object sender, RoutedEventArgs e)
{
    btnContinuar.IsEnabled = true;
}

private void CheckBox_Unchecked(object sender, RoutedEventArgs e)
{
    btnContinuar.IsEnabled = false;
}
```

Atributos comunes

- Content: Texto mostrado.
- IsChecked: Estado del CheckBox.
- Margin, Padding: Espaciados exteriores e interiores.
- Width, Height: Tamaño del control.
- Foreground: Color del texto.

2.4 <RadioButton>

¿Qué es?

Control para seleccionar una única opción dentro de un grupo.

¿Para qué se usa?

Permitir selección exclusiva entre varias opciones.

¿Dónde es mejor usarlo?

Formularios de selección, encuestas, configuraciones exclusivas.

¿Cuándo evitarlo?

Evitar si múltiples opciones pueden seleccionarse (usar CheckBox).

Ejemplo básico en XAML

```
<RadioButton
    Content="Opción A"
    GroupName="Opciones"
    Width="150"
    Height="30"
    FontSize="13"
    FontFamily="Verdana"
    Margin="10"/>
```

Ejemplo avanzado en XAML

```
<StackPanel Margin="20">
    <RadioButton Name="rbMasculino" Content="Masculino" GroupName="Genero"
    Margin="0,5"/>
    <RadioButton Name="rbFemenino" Content="Femenino" GroupName="Genero"
    Margin="0,5"/>
    <Button Content="Confirmar" Width="100" Margin="0,10"
    Click="ButtonConfirmarGenero_Click"/>
</StackPanel>
```

Ejemplo en code-behind

```
private void ButtonConfirmarGenero_Click(object sender, RoutedEventArgs e)
{
    if (rbMasculino.IsChecked == true)
```



```

        MessageBox.Show("Has seleccionado Masculino.");    else if (rbFemenino.IsChecked ==
true)
        MessageBox.Show("Has seleccionado Femenino.");    else
        MessageBox.Show("No has seleccionado ningún género.");
    }

```

Atributos comunes

- Content: Texto de la opción.
- GroupName: Nombre del grupo de exclusividad.
- IsChecked: Estado de selección.
- Margin: Espaciado exterior.
- Width, Height: Tamaño.

2.5 <ComboBox>

¿Qué es?

Lista desplegable que permite seleccionar un ítem de varias opciones.

¿Para qué se usa?

Ofrecer múltiples opciones de selección en un espacio reducido.

¿Dónde es mejor usarlo?

Formularios de selección de país, categoría, opciones de configuración.

¿Cuándo evitarlo?

Evitar si es necesario mostrar todas las opciones a la vez (usar ListBox).

Ejemplo básico en XAML

```

<ComboBox
    Width="200"
    Height="30"
    FontSize="13"
    Margin="10"
    Background="WhiteSmoke"
    Foreground="DarkSlateGray">
    <ComboBoxItem Content="España"/>
    <ComboBoxItem Content="México"/>
    <ComboBoxItem Content="Argentina"/>
</ComboBox>

```

Ejemplo avanzado en XAML

```
<StackPanel Margin="20">
    <ComboBox Name="cbPaises" Width="200"
SelectionChanged="ComboBoxPaises_SelectionChanged" Margin="0,5">
        <ComboBoxItem Content="España"/>
        <ComboBoxItem Content="México"/>
        <ComboBoxItem Content="Argentina"/>
    </ComboBox>
    <TextBlock Name="txtSeleccion" FontWeight="Bold" Margin="0,10"/>
</StackPanel>
```

Ejemplo en code-behind

```
private void ComboBoxPaises_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    if (cbPaises.SelectedItem is ComboBoxItem item)
    {
        txtSeleccion.Text = $"País seleccionado: {item.Content}";
    }
}
```

Atributos comunes

- Items: Elementos dentro del ComboBox.
- SelectedItem: Elemento seleccionado.
- SelectedIndex: Índice seleccionado.
- Margin: Espaciado exterior.
- Width, Height: Tamaño del control.

.

2.6 <ListBox>

¿Qué es?

Control que muestra una lista de elementos de los cuales uno o varios pueden ser seleccionados.

¿Para qué se usa?

Mostrar listas visibles de opciones para que el usuario seleccione una o varias.

¿Dónde es mejor usarlo?

Menús, listas de reproducción, selección de elementos múltiples.

¿Cuándo evitarlo?

Evitar si quieres ahorrar espacio mostrando solo una opción (usar ComboBox).

Ejemplo básico en XAML

```
<ListBox
  Width="200"
  Height="120"
  Background="White"
  Foreground="Black"
  FontSize="13"
  FontFamily="Calibri"
  Margin="10">
  <ListBoxItem Content="Opción 1"/>
  <ListBoxItem Content="Opción 2"/>
  <ListBoxItem Content="Opción 3"/>
</ListBox>
```

Ejemplo avanzado en XAML

```
<StackPanel Margin="20">
  <ListBox Name="lstFrutas" Width="200" Height="100"
    SelectionChanged="ListBox_SelectionChanged">
    <ListBoxItem Content="Manzana"/>
    <ListBoxItem Content="Banana"/>
    <ListBoxItem Content="Naranja"/>
  </ListBox>
  <TextBlock Name="txtSeleccionFruta" FontWeight="Bold" Margin="0,10"/>
</StackPanel>
```

Ejemplo en code-behind

```
private void ListBox_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    if (lstFrutas.SelectedItem is ListBoxItem item)
    {
        txtSeleccionFruta.Text = $"Fruta seleccionada: {item.Content}";
    }
}
```

```
}  
}
```

Atributos comunes

- Items: Lista de elementos dentro del ListBox.
- SelectedItem: Elemento actualmente seleccionado.
- SelectedIndex: Índice del elemento seleccionado.
- SelectionMode: Modo de selección (Single, Multiple, Extended).
- Margin: Espaciado exterior.
- Width, Height: Dimensiones del ListBox.

2.7 <Slider>

¿Qué es?

Control deslizante para seleccionar un valor dentro de un rango.

¿Para qué se usa?

Permitir al usuario seleccionar un valor numérico de forma intuitiva.

¿Dónde es mejor usarlo?

Volumen, brillo, escalas de valores, configuraciones de rango.

¿Cuándo evitarlo?

Evitar si se necesita introducir valores exactos mediante teclado (usar TextBox).

Ejemplo básico en XAML

```
<Slider  
    Width="200"  
    Height="30"  
    Minimum="0"  
    Maximum="100"  
    Value="50"  
    Background="LightGray"  
    Foreground="DarkSlateGray"  
    Margin="10"/>
```

Ejemplo avanzado en XAML

```
<StackPanel Margin="20">  
    <Slider Name="sliderVolumen" Minimum="0" Maximum="100"  
    ValueChanged="SliderVolumen_ValueChanged" Width="200"/>
```

```
<TextBlock Name="txtVolumen" Margin="0,10"/>
</StackPanel>
```

Ejemplo en code-behind

```
private void SliderVolumen_ValueChanged(object sender,
RoutedPropertyChangedEventArgs<double> e)
{
    txtVolumen.Text = $"Volumen: {sliderVolumen.Value:F0}%";
}
```

Atributos comunes

- Minimum: Valor mínimo permitido.
- Maximum: Valor máximo permitido.
- Value: Valor actual del slider.
- TickFrequency: Intervalo entre marcas de valor.
- Width, Height: Dimensiones.
- Margin: Espaciado exterior.

2.9 <ProgressBar>

¿Qué es?

Control visual para representar el progreso de una tarea.

¿Para qué se usa?

Indicar al usuario el estado de avance de una operación.

¿Dónde es mejor usarlo?

Carga de archivos, descargas, procesos largos.

¿Cuándo evitarlo?

Evitar si no es necesario mostrar el avance visualmente.

Ejemplo básico en XAML

```
<ProgressBar
    Width="200"
    Height="20"
    Minimum="0"
    Maximum="100"
```

```
Value="30"  
Background="White"  
Foreground="Green"  
Margin="10"/>
```

Ejemplo avanzado en XAML

```
<StackPanel Margin="20">  
    <ProgressBar Name="progressBar" Minimum="0" Maximum="100" Width="200"  
Height="20" Margin="0,5"/>  
    <Button Content="Avanzar" Click="ButtonAvanzar_Click" Width="100" Margin="0,10"/>  
    <Label Content="Cada vez que clicas avanzo"  
HorizontalContentAlignment="Center"></Label>  
</StackPanel>
```

Ejemplo en code-behind

```
private void ButtonAvanzar_Click(object sender, RoutedEventArgs e)  
{  
    if (progressBar.Value < 100)    progressBar.Value += 10;  
}
```

Atributos comunes

- Minimum: Valor mínimo del progreso.
- Maximum: Valor máximo del progreso.
- Value: Valor actual del progreso.
- Foreground: Color de la barra de progreso.
- Width, Height: Dimensiones del control.

2.10 <PasswordBox>

¿Qué es?

Control de entrada de texto donde los caracteres se ocultan para seguridad.

¿Para qué se usa?

Permitir introducir contraseñas de manera segura.

¿Dónde es mejor usarlo?

Pantallas de inicio de sesión, formularios de registro.

¿Cuándo evitarlo?

Evitar si el texto no es confidencial (usar TextBox normal).

Ejemplo básico en XAML

```
<PasswordBox
    Width="200"
    Height="30"
    FontSize="13"
    Margin="10"
    Background="White"
    Foreground="Black"/>
```

Ejemplo avanzado en XAML

```
<StackPanel Margin="20">
    <PasswordBox Name="pwdClave" Width="200" Height="30" Margin="0,5"/>
    <Button Content="Verificar" Width="100" Margin="0,10" Click="ButtonVerificar_Click"/>
</StackPanel>
```

Ejemplo en code-behind

```
private void ButtonVerificar_Click(object sender, RoutedEventArgs e)
{
    if (pwdClave.Password == "1234")    MessageBox.Show("Acceso concedido."); else
        MessageBox.Show("Clave incorrecta.");
}
```

Atributos comunes

- Password: Contraseña ingresada.
- MaxLength: Longitud máxima de la contraseña.
- Width, Height: Dimensiones del control.
- Margin: Espaciado exterior.

2.11 <Image>

¿Qué es?

Control que muestra una imagen gráfica.

¿Para qué se usa?

Mostrar logos, fotos, ilustraciones dentro de la interfaz.

¿Dónde es mejor usarlo?

Cabeceras, perfiles de usuario, galerías de imágenes.

¿Cuándo evitarlo?

Evitar si necesitas solo mostrar texto (usar TextBlock o Label).

Ejemplo básico en XAML

```
<Image
  Source="logo.png"
  Width="150"
  Height="150"
  Stretch="Uniform"
  Margin="10"/>
```

Ejemplo avanzado en XAML

```
<StackPanel Margin="20">
  <Image Name="imgLogo" Width="150" Height="150" Stretch="Uniform"
  Source="logo.png"/>
</StackPanel>
```

Ejemplo en code-behind

// No requiere code-behind para mostrar una imagen estática.

Atributos comunes

- Source: Ruta o recurso de la imagen.
- Stretch: Cómo se adapta la imagen.
- Width, Height: Dimensiones.
- Margin: Espaciado exterior.

2.12 <TextBlock>

¿Qué es?

Control ligero de presentación de texto no editable.

¿Para qué se usa?

Mostrar mensajes, títulos, instrucciones o pequeños fragmentos de texto en la interfaz.

¿Dónde es mejor usarlo?

Pantallas informativas, formularios, encabezados de sección, mensajes de ayuda.

¿Cuándo evitarlo?

Evitar si el usuario necesita introducir o editar texto (usar TextBox).

Ejemplo básico en XAML

```
<TextBlock
  Text="Bienvenido a la aplicación"
  Width="300"
  Height="50"
  Background="LightGray"
  Foreground="DarkGreen"
  FontSize="16"
  FontWeight="Bold"
  FontFamily="Segoe UI"
  TextAlignment="Center"
  Margin="10"
  Padding="5"/>
```

Ejemplo avanzado en XAML

```
<StackPanel Margin="20">
  <TextBlock Name="txtMensaje" FontSize="16" FontWeight="Bold"
    TextAlignment="Center" Margin="10"/>
  <Button Content="Mostrar Mensaje" Width="150" Click="ButtonMostrarMensaje_Click"/>
</StackPanel>
```

Ejemplo en code-behind

```
private void ButtonMostrarMensaje_Click(object sender, RoutedEventArgs e)
{
    txtMensaje.Text = "¡Gracias por visitar nuestra aplicación!";
}
```

Atributos comunes

- Text: Contenido del texto mostrado.
- FontSize, FontFamily, FontWeight: Estilos de fuente.
- Foreground: Color del texto.
- TextAlignment: Alineación del texto (Left, Center, Right, Justify).
- TextWrapping: Permite que el texto se ajuste en múltiples líneas.
- Margin, Padding: Espaciados exteriores e interiores.
- Width, Height: Dimensiones del TextBlock.

3. Controles Avanzados

3.1 <TabControl y TabItem>

¿Qué es?

TabControl es un contenedor que permite mostrar contenido organizado en pestañas. Cada pestaña se define con un TabItem.

¿Para qué se usa?

Organizar múltiples secciones de información o formularios en un espacio compacto utilizando pestañas.

¿Dónde es mejor usarlo?

Interfaces que requieren agrupar datos relacionados pero mantener la interfaz limpia (configuraciones, formularios extensos, áreas de navegación).

¿Cuándo evitarlo?

Evitar cuando hay muy poca información para justificar múltiples pestañas o si el contenido debe ser visto de manera simultánea.

Ejemplo básico en XAML

```
<TabControl
    Width="400"
    Height="200"
```

```

    Background="LightGray"
    Margin="10">
<TabItem Header="General">
    <TextBlock Text="Contenido General" Padding="10"/>
</TabItem>
<TabItem Header="Detalles">
    <TextBlock Text="Contenido Detallado" Padding="10"/>
</TabItem>
</TabControl>

```

Ejemplo avanzado en XAML

```

<Grid Margin="20">
<TabControl Name="tabPrincipal" Width="400" Height="250">
<TabItem Header="Inicio">
<StackPanel Margin="10">
    <TextBlock Text="Bienvenido a la pestaña Inicio" Margin="0,5"/>
    <Button Content="Acción Inicio" Click="ButtonAccionInicio_Click" Margin="0,10"/>
</StackPanel>
</TabItem>
<TabItem Header="Configuración">
<StackPanel Margin="10">
    <TextBlock Text="Configuración de la Aplicación" Margin="0,5"/>
    <Button Content="Guardar Configuración" Click="ButtonGuardarConfiguracion_Click"
Margin="0,10"/>
</StackPanel>
</TabItem>
</TabControl>
</Grid>

```

Ejemplo en code-behind

```

private void ButtonAccionInicio_Click(object sender, RoutedEventArgs e)
{
    MessageBox.Show("Has hecho clic en la pestaña Inicio.");
}

private void ButtonGuardarConfiguracion_Click(object sender, RoutedEventArgs e)
{
    MessageBox.Show("Configuración guardada correctamente.");
}

```

Atributos comunes

- TabStripPlacement: Coloca las pestañas arriba, abajo, izquierda o derecha.
- SelectedIndex: Índice del TabItem seleccionado actualmente.
- SelectedItem: TabItem seleccionado actualmente.
- Margin, Padding: Espaciado exterior e interior.
- Background, Foreground: Colores de fondo y texto.
- Width, Height: Dimensiones del TabControl.

3.2 <GroupBox>

¿Qué es?

Contenedor que agrupa controles relacionados dentro de un borde con un título.

¿Para qué se usa?

Organizar visualmente conjuntos de controles relacionados en formularios y configuraciones.

¿Dónde es mejor usarlo?

Formularios largos, secciones de configuración, diálogos de opciones.

¿Cuándo evitarlo?

Evitar si solo hay un único control dentro (usar un simple Border o StackPanel).

Ejemplo básico en XAML

```
<GroupBox
  Header="Datos Personales"
  Width="300"
  Height="200"
  FontSize="14"
  FontWeight="Bold"
  Margin="10">
  <StackPanel Margin="10">
    <TextBlock Text="Nombre:"/>
    <TextBox Width="200"/>
    <TextBlock Text="Edad:" Margin="0,10,0,0"/>
    <TextBox Width="100"/>
  </StackPanel>
</GroupBox>
```

Ejemplo avanzado en XAML

```

<Grid Margin="20">
  <GroupBox Header="Información de Contacto" Width="350" Height="250">
    <StackPanel Margin="10">
      <Label Content="Email:"/>
      <TextBox Name="txtEmail" Width="250"/>
      <Button Content="Enviar" Width="100" Margin="0,10" Click="ButtonEnviar_Click"/>
    </StackPanel>
  </GroupBox>
</Grid>

```

Ejemplo en code-behind

```

private void ButtonEnviar_Click(object sender, RoutedEventArgs e)
{
    MessageBox.Show($"Email enviado a: {txtEmail.Text}");
}

```

Atributos comunes

- Header: Título del GroupBox.
- Content: Contenido interno.
- Width, Height: Dimensiones.
- Margin, Padding: Espaciados exteriores e interiores.
- FontSize, FontWeight: Estilos del título.

3.3 <Expander>

¿Qué es?

Contenedor que puede expandirse o colapsarse para mostrar u ocultar su contenido.

¿Para qué se usa?

Optimizar espacio mostrando información adicional solo cuando sea necesario.

¿Dónde es mejor usarlo?

Paneles de ayuda, formularios con opciones avanzadas.

¿Cuándo evitarlo?

Evitar si toda la información es crítica (en ese caso, mostrar siempre).

Ejemplo básico en XAML

```
<Expander
  Header="Opciones Avanzadas"
  Width="300"
  Background="LightGray"
  FontSize="13"
  Margin="10">
  <StackPanel Margin="10">
    <CheckBox Content="Habilitar notificaciones"/>
    <CheckBox Content="Activar modo oscuro"/>
  </StackPanel>
</Expander>
```

Ejemplo avanzado en XAML

```
<StackPanel Margin="20">
  <Expander Header="Más Configuraciones" Width="300">
    <StackPanel Margin="10">
      <TextBlock Text="Opción 1"/>
      <Button Content="Activar" Click="ButtonActivar_Click"/>
    </StackPanel>
  </Expander>
</StackPanel>
```

Ejemplo en code-behind

```
private void ButtonActivar_Click(object sender, RoutedEventArgs e)
{
    MessageBox.Show("Opción activada correctamente.");
}
```

Atributos comunes

- Header: Título visible del Expander.
- IsExpanded: Estado inicial (expandido o colapsado).
- Width, Height: Dimensiones.
- Margin: Espaciado exterior.
- Content: Contenido mostrado al expandir.

3.4 <ScrollView>

¿Qué es?

Contenedor que permite desplazamiento vertical y horizontal de su contenido.

¿Para qué se usa?

Agregar scroll a elementos cuyo contenido puede exceder el espacio disponible.

¿Dónde es mejor usarlo?

Textos largos, listas grandes, imágenes grandes.

¿Cuándo evitarlo?

Evitar si todo el contenido cabe perfectamente en la pantalla.

Ejemplo básico en XAML

```
<ScrollView
    Width="100"
    Height="50"
    VerticalScrollBarVisibility="Auto"
    Margin="10">
    <TextBlock Text="Texto muy largo que necesita scroll para ser visualizado
completamente..."
        TextWrapping="Wrap"/>
</ScrollView>
```

Ejemplo avanzado en XAML

```
<Grid Margin="20">
    <ScrollView Width="300" Height="150" HorizontalScrollBarVisibility="Auto">
        <StackPanel Orientation="Horizontal">
            <Button Content="Elemento 1" Width="100" Margin="5"/>
            <Button Content="Elemento 2" Width="100" Margin="5"/>
            <Button Content="Elemento 3" Width="100" Margin="5"/>
        </StackPanel>
    </ScrollView>
</Grid>
```

Ejemplo en code-behind * ScrollView maneja automáticamente el scroll, no necesita eventos en este ejemplo.

Atributos comunes

- HorizontalScrollBarVisibility: Visibilidad de la barra horizontal.

- VerticalScrollBarVisibility: Visibilidad de la barra vertical.
- Content: Contenido desplazable.
- Width, Height: Dimensiones del área visible.
- Margin: Espaciado exterior.

3.6 <Menu> y <MenuItem>

¿Qué es?

Contenedor para menús de navegación y sus opciones individuales.

¿Para qué se usa?

Ofrecer navegación estructurada y opciones de comandos en aplicaciones.

¿Dónde es mejor usarlo?

Aplicaciones de escritorio que necesitan menús superiores (Archivo, Edición, etc.).

¿Cuándo evitarlo?

Evitar en interfaces extremadamente simples (podría ser más confuso que útil).

Ejemplo básico en XAML

```
<Menu Margin="10">
  <MenuItem Header="Archivo">
    <MenuItem Header="Nuevo"/>
    <MenuItem Header="Abrir"/>
    <MenuItem Header="Salir"/>
  </MenuItem>
  <MenuItem Header="Ayuda">
    <MenuItem Header="Acerca de"/>
  </MenuItem>
</Menu>
```

Ejemplo avanzado en XAML

```
<Grid Margin="20">
  <Menu>
    <MenuItem Header="Opciones">
      <MenuItem Header="Configuración" Click="MenuItemConfiguracion_Click"/>
      <MenuItem Header="Salir" Click="MenuItemSalir_Click"/>
    </MenuItem>
  </Menu>
</Grid>
```


Ejemplo en code-behind

```
private void MenuItemConfiguracion_Click(object sender, RoutedEventArgs e)
{
    MessageBox.Show("Configuración abierta.");
}

private void MenuItemSalir_Click(object sender, RoutedEventArgs e)
{
    Application.Current.Shutdown();
}
```

Atributos comunes

- Header: Texto del elemento de menú.
- Items: Submenús o acciones dentro de un menú.
- IsEnabled: Habilita o deshabilita el elemento.
- Margin: Espaciado exterior.

3.7 <ToolBar>

¿Qué es?

Contenedor para un conjunto de botones de acceso rápido a funciones.

¿Para qué se usa?

Proveer accesos directos a las funciones más comunes.

¿Dónde es mejor usarlo?

Aplicaciones de edición, herramientas de gestión, aplicaciones de oficina.

¿Cuándo evitarlo?

Evitar si se tienen muy pocas acciones (mejor solo botones normales).

Ejemplo básico en XAML

```
<ToolBar Margin="10">
    <Button Content="Nuevo"/>
    <Button Content="Guardar"/>
    <Button Content="Imprimir"/>
</ToolBar>
```

Ejemplo avanzado en XAML

```
<Grid Margin="20">
  <ToolBarTray>
    <ToolBar>
      <Button Content="Nuevo" Click="ButtonNuevo_Click"/>
      <Button Content="Guardar" Click="ButtonGuardar_Click"/>
    </ToolBar>
  </ToolBarTray>
</Grid>
```

Ejemplo en code-behind

```
private void ButtonNuevo_Click(object sender, RoutedEventArgs e)
{
    MessageBox.Show("Nuevo documento creado.");
}

private void ButtonGuardar_Click(object sender, RoutedEventArgs e)
{
    MessageBox.Show("Documento guardado.");
}
```

Atributos comunes

- Items: Elementos dentro de la barra.
- Orientation: Orientación horizontal o vertical.
- IsOverflowOpen: Muestra el contenido adicional que no cabe.
- Margin: Espaciado exterior.

3.8 <StatusBar>

¿Qué es?

Barra inferior que muestra información de estado o mensajes de la aplicación.

¿Para qué se usa?

Dar retroalimentación al usuario sobre el estado de la aplicación.

¿Dónde es mejor usarlo?

Aplicaciones de escritorio que necesiten mostrar mensajes dinámicos o estados.

¿Cuándo evitarlo?

Evitar en interfaces extremadamente simples o móviles.

Ejemplo básico en XAML

```
<StatusBar Margin="10">
  <StatusBarItem Content="Listo"/>
  <StatusBarItem Content="Usuario: admin"/>
</StatusBar>
```

Ejemplo avanzado en XAML

```
<Grid Margin="20">
  <Grid.RowDefinitions>
    <RowDefinition Height="*" />
    <RowDefinition Height="Auto" />
  </Grid.RowDefinitions>

  <Button Content="Actualizar Estado"
    Grid.Row="0"
    HorizontalAlignment="Center"
    VerticalAlignment="Center"
    Click="ButtonActualizarEstado_Click"
    Margin="10" />

  <StatusBar Grid.Row="1">
    <StatusBarItem>
      <TextBlock x:Name="txtEstado" Text="Esperando acción..." />
    </StatusBarItem>
  </StatusBar>
</Grid>
```

Ejemplo en code-behind

```
private void ButtonActualizarEstado_Click(object sender, RoutedEventArgs e)
{
    txtEstado.Text = "Procesando solicitud...";
}
```

Atributos comunes

- Items: Elementos dentro de la barra de estado.
- Height: Altura del StatusBar.
- Margin: Espaciado exterior.
- Background: Color de fondo.

3.9 <TreeView>

¿Qué es?

Control que presenta información en forma de estructura jerárquica (como carpetas y archivos).

¿Para qué se usa?

Visualizar relaciones padre-hijo de forma clara.

¿Dónde es mejor usarlo?

Exploradores de archivos, categorías anidadas, menús jerárquicos.

¿Cuándo evitarlo?

Evitar si los datos son planos o no tienen jerarquías.

Ejemplo básico en XAML

```
<TreeView Width="300" Height="200" Margin="10">
  <TreeViewItem Header="Animales">
    <TreeViewItem Header="Mamíferos"/>
    <TreeViewItem Header="Aves"/>
    <TreeViewItem Header="Reptiles"/>
  </TreeViewItem>
  <TreeViewItem Header="Plantas">
    <TreeViewItem Header="Árboles"/>
    <TreeViewItem Header="Flores"/>
  </TreeViewItem>
</TreeView>
```

Ejemplo avanzado en XAML

```
<StackPanel Margin="20">
  <TreeView Name="treeCategorias" Width="300" Height="200"
    SelectedItemChanged="TreeView_SelectedItemChanged">
    <TreeViewItem Header="Tecnología">
```

```

        <TreeViewItem Header="Computadoras"/>
        <TreeViewItem Header="Smartphones"/>
    </TreeViewItem>
    <TreeViewItem Header="Hogar">
        <TreeViewItem Header="Cocina"/>
        <TreeViewItem Header="Sala"/>
    </TreeViewItem>
</TreeView>
<TextBlock Name="txtCategoria" Margin="10"/>
</StackPanel>

```

Ejemplo en code-behind

```

private void TreeView_SelectedItemChanged(object sender,
RoutedPropertyChangedEventArgs<object> e)
{
    if (treeCategorias.SelectedItem is TreeViewItem item)
    {
        txtCategoria.Text = $"Categoría seleccionada: {item.Header}";
    }
}

```

Atributos comunes

- Items: Elementos dentro del árbol.
- SelectedItem: Elemento actualmente seleccionado.
- SelectedItemChanged: Evento al seleccionar un nodo.
- Width, Height: Dimensiones.
- Margin: Espaciado exterior.

3.10 <ListView>

¿Qué es?

Control para mostrar listas de datos en formato de filas, con soporte para plantillas y múltiples columnas.

¿Para qué se usa?

Presentar información detallada en forma de lista o cuadrícula.

¿Dónde es mejor usarlo?

Listados de usuarios, catálogos de productos, registros de eventos.

¿Cuándo evitarlo?

Evitar si basta una lista simple de elementos sin detalle (usar ListBox).

Ejemplo básico en XAML

```
<ListView Width="300" Height="200" Margin="10">
  <ListViewItem Content="Item 1"/>
  <ListViewItem Content="Item 2"/>
  <ListViewItem Content="Item 3"/>
</ListView>
```

Atributos comunes

- ItemsSource: Fuente de datos de la lista.
- SelectedItem: Elemento seleccionado.
- View: Vista para mostrar las columnas (GridView).
- Width, Height: Dimensiones.
- Margin: Espaciado exterior.

3.11 <DataGrid>

¿Qué es?

Control que presenta datos tabulares editables y permite ordenación, agrupación y navegación.

¿Para qué se usa?

Visualizar y editar conjuntos de datos en forma de tabla.

¿Dónde es mejor usarlo?

Aplicaciones administrativas, bases de datos, sistemas de inventarios.

¿Cuándo evitarlo?

Evitar si solo necesitas mostrar información básica sin edición ni estructura tabular.

Ejemplo básico en XAML

```
<DataGrid Width="400" Height="200" Margin="10" AutoGenerateColumns="True"/>
```

Ejemplo avanzado en XAML

```
<StackPanel Margin="20">
```

```

<DataGrid Name="dataUsuarios" AutoGenerateColumns="False" Width="400"
Height="200" SelectionChanged="DataGrid_SelectionChanged">
  <DataGrid.Columns>
    <DataGridTextColumn Header="Nombre" Binding="{Binding Nombre}"/>
    <DataGridTextColumn Header="Edad" Binding="{Binding Edad}"/>
  </DataGrid.Columns>
</DataGrid>
<TextBlock Name="txtUsuarioSeleccionado" Margin="10"/>
</StackPanel>

```

Atributos comunes

- ItemsSource: Fuente de datos.
- AutoGenerateColumns: Crear columnas automáticamente.
- SelectedItem: Elemento seleccionado.
- Columns: Definición manual de columnas.
- Width, Height: Dimensiones.

3.12 <Calendar>

¿Qué es?

Control que muestra un calendario completo para seleccionar fechas.

¿Para qué se usa?

Permitir a los usuarios seleccionar fechas de forma visual e intuitiva.

¿Dónde es mejor usarlo?

Formularios de reservas, agendas, calendarios personales.

¿Cuándo evitarlo?

Evitar si basta una simple caja de texto para fecha (usar DatePicker).

Ejemplo básico en XAML

```

<Calendar Width="300" Height="300" Margin="10"/>

```

Ejemplo avanzado en XAML

```

<StackPanel Margin="20">
  <Calendar Name="calendarEventos"
SelectedDatesChanged="Calendar_SelectedDatesChanged" Width="300" Height="300"/>
  <TextBlock Name="txtFechaSeleccionada" Margin="10"/>

```

</StackPanel>

Ejemplo en code-behind

```
private void Calendar_SelectedDatesChanged(object sender, SelectionChangedEventArgs e)
{
    if (calendarEventos.SelectedDate.HasValue)
    {
        txtFechaSeleccionada.Text = $"Fecha seleccionada:
{calendarEventos.SelectedDate.Value.ToShortDateString()}";
    }
}
```

Atributos comunes

- SelectedDate: Fecha seleccionada.
- DisplayDate: Fecha mostrada actualmente.
- SelectionMode: Modo de selección (Single, MultipleRange, etc.).
- Width, Height: Dimensiones.
- Margin: Espaciado exterior.

3.13 <DatePicker>

¿Qué es?

Control para seleccionar una fecha usando un cuadro combinado y un calendario desplegable.

¿Para qué se usa?

Facilitar la selección de fechas de forma compacta.

¿Dónde es mejor usarlo?

Formularios de entrada de datos, selección de fechas rápidas.

¿Cuándo evitarlo?

Evitar si se necesita una visualización constante de muchos días (usar Calendar).

Ejemplo básico en XAML

```
<DatePicker Width="200" Margin="10"/>
```


Ejemplo avanzado en XAML

```
<StackPanel Margin="20">
    <DatePicker Name="dateNacimiento"
SelectedDateChanged="DatePicker_SelectedDateChanged" Width="200"/>
    <TextBlock Name="txtNacimiento" Margin="10"/>
</StackPanel>
```

Ejemplo en code-behind

```
private void DatePicker_SelectedDateChanged(object sender, SelectionChangedEventArgs e)
{
    if (dateNacimiento.SelectedDate.HasValue)
    {
        txtNacimiento.Text = $"Fecha de nacimiento:
{dateNacimiento.SelectedDate.Value.ToShortDateString()}";
    }
}
```

Atributos comunes

- SelectedDate: Fecha seleccionada.
- DisplayDateStart, DisplayDateEnd: Rango de fechas disponibles.
- Width, Height: Dimensiones.
- Margin: Espaciado exterior.

3.14 <Frame>

¿Qué es?

Contenedor que permite mostrar diferentes páginas (Page) dentro de una misma ventana.

¿Para qué se usa?

Gestionar navegación entre diferentes contenidos sin cambiar toda la ventana.

¿Dónde es mejor usarlo?

Aplicaciones que necesiten navegación interna entre secciones (apps de configuración, tutoriales, asistentes).

¿Cuándo evitarlo?

Evitar en aplicaciones muy simples que no requieren navegación por páginas.

Ejemplo básico en XAML

```
<Frame
    Width="400"
    Height="300"
    Margin="10"
    NavigationUIVisibility="Visible"
    Source="Page1.xaml"/>
```

Ejemplo avanzado en XAML

```
<Grid Margin="20">
    <StackPanel>
        <Button Content="Ir a Página 1" Click="ButtonPagina1_Click" Width="150"
Margin="0,0,0,10"/>
        <Button Content="Ir a Página 2" Click="ButtonPagina2_Click" Width="150"
Margin="0,0,0,10"/>
        <Frame Name="frameNavegacion" Width="400" Height="300"
NavigationUIVisibility="Visible"/>
    </StackPanel>
</Grid>
```

Ejemplo en code-behind **Hay que crear Page1 y Page2, explicado más adelante**

```
private void ButtonPagina1_Click(object sender, RoutedEventArgs e)
{
    frameNavegacion.Navigate(new Uri("Page1.xaml", UriKind.Relative));
}
```

```
private void ButtonPagina2_Click(object sender, RoutedEventArgs e)
{
    frameNavegacion.Navigate(new Uri("Page2.xaml", UriKind.Relative));
}
```

Atributos comunes

- Source: Página a cargar inicialmente.
- NavigationUIVisibility: Mostrar u ocultar botones de navegación.
- Width, Height: Dimensiones del Frame.
- Margin: Espaciado exterior.

3.15 <Page>

¿Qué es?

Representa una página de contenido que puede ser navegada dentro de un Frame o NavigationWindow.

¿Para qué se usa?

Definir diferentes secciones independientes de contenido en una aplicación.

¿Dónde es mejor usarlo?

Aplicaciones divididas en múltiples pasos, formularios extensos, documentación navegable.

¿Cuándo evitarlo?

Evitar si no se requiere cambiar de contenido dinámicamente (usar UserControl si es estático).

Ejemplo básico en XAML

```
<Page x:Class="MiAplicacion.Page1"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Página 1">
    <Grid Background="LightBlue">
        <TextBlock Text="Esta es la Página 1" HorizontalAlignment="Center"
VerticalAlignment="Center" FontSize="20"/>
    </Grid>
</Page>
```

Ejemplo avanzado en XAML

```
<Page x:Class="MiAplicacion.Page2"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Página 2">
    <StackPanel Margin="20">
        <TextBlock Text="Bienvenido a la Página 2" FontSize="18" Margin="0,0,0,20"/>
        <Button Content="Ir a Página 1" Click="ButtonVolver_Click" Width="150"/>
    </StackPanel>
</Page>
```

Ejemplo en code-behind

```
private void ButtonVolver_Click(object sender, RoutedEventArgs e)
{
    this.NavigationService.Navigate(new Uri("Page1.xaml", UriKind.Relative));
}
```

Atributos comunes

- Title: Título de la página.
- NavigationService: Permite controlar la navegación desde la página.
- Content: Contenido de la página.
- Background: Fondo de la página.

3.16 <NavigationWindow>

¿Qué es?

Ventana especializada en navegación entre páginas, con botones de avance y retroceso integrados.

¿Para qué se usa?

Simplificar la gestión de navegación, manejando historial automáticamente.

¿Dónde es mejor usarlo?

Aplicaciones que simulan una navegación estilo navegador web.

¿Cuándo evitarlo?

Evitar en apps que solo necesitan cambiar pequeñas secciones internas (usar Frame en su lugar).

Ejemplo básico en XAML

```
<NavigationWindow
    Source="Page1.xaml"
    ShowsNavigationUI="True"
    Width="600"
    Height="400"
    WindowStartupLocation="CenterScreen"
    Title="Aplicación de Navegación"/>
```

Ejemplo avanzado en XAML

```
<!-- Normalmente se define en App.xaml o en el arranque de la aplicación -->
<Application x:Class="MiAplicacion.App"      StartupUri="NavigationWindow.xaml"
            xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
            xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
</Application>
```

Ejemplo en code-behind

// La navegación se gestiona automáticamente. No es necesario manejar eventos básicos.

Atributos comunes

- Source: Página inicial que carga.
- ShowsNavigationUI: Mostrar u ocultar barra de navegación.
- Width, Height: Dimensiones de la ventana.
- Title: Título de la ventana.

3.17 <Hyperlink>

¿Qué es?

Elemento interactivo dentro de un TextBlock que permite navegar a otra página o URL.

¿Para qué se usa?

Crear enlaces de navegación en el texto, similar a los de una página web.

¿Dónde es mejor usarlo?

Mensajes de ayuda, documentación, navegación contextual dentro de la app.

¿Cuándo evitarlo?

Evitar en interfaces donde el usuario no espera elementos clicables dentro de texto.

Ejemplo básico en XAML

```
<TextBlock Margin="10">
    Para más información, visita
    <Hyperlink NavigateUri="https://www.example.com"
        RequestNavigate="Hyperlink_RequestNavigate"> nuestro sitio web </Hyperlink>.
</TextBlock>
```

Ejemplo avanzado en XAML

```

<StackPanel Margin="20">
  <TextBlock>
    Ir a
    <Hyperlink NavigateUri="https://www.microsoft.com"
RequestNavigate="Hyperlink_RequestNavigate">
      Microsoft </Hyperlink>.
  </TextBlock>
</StackPanel>

```

Ejemplo en code-behind

```

private void Hyperlink_RequestNavigate(object sender, RequestNavigateEventArgs e)
{
    System.Diagnostics.Process.Start(new ProcessStartInfo(e.Uri.AbsoluteUri) {
UseShellExecute = true }); e.Handled = true;
}

```

Atributos comunes

- **NavigateUri:** URI a la que se navega.
- **RequestNavigate:** Evento que maneja la navegación.
- **Inlines:** Contenido de texto combinado dentro de un TextBlock.
- **Foreground:** Color del enlace.
- **TextDecorations:** Decoraciones de texto como subrayado.

4. Estilos, Gráficos y Efectos

Esta sección describe los controles de formas gráficas y transformaciones en WPF, incluyendo ejemplos básicos elegantes, ejemplos avanzados y listado de atributos comunes.

4.1 <Ellipse>

¿Qué es?

Control para dibujar una elipse o un círculo en la interfaz de usuario.

¿Para qué se usa?

Crear formas circulares decorativas, botones redondos, gráficos visuales.

¿Dónde es mejor usarlo?

Aplicaciones gráficas, dashboards, decoraciones de interfaz.

¿Cuándo evitarlo?

Evitar si necesitas interacción compleja (usar un control compuesto).

Ejemplo básico en XAML

```
<Ellipse
    Width="100"
    Height="100"
    Fill="LightBlue"
    Stroke="SlateBlue"
    StrokeThickness="3"
    Margin="10"/>
```

Ejemplo avanzado en XAML

```
<Grid Margin="20">
    <Ellipse Width="150" Height="150">
        <Ellipse.Fill>
            <RadialGradientBrush>
                <GradientStop Color="Lavender" Offset="0.0"/>    <GradientStop Color="SlateBlue"
Offset="1.0"/>
            </RadialGradientBrush>
        </Ellipse.Fill>
    </Ellipse>
</Grid>
```

Atributos comunes

- Fill: Relleno de la elipse.
- Stroke: Color del borde.
- StrokeThickness: Grosor del borde.
- Width, Height: Dimensiones.
- Margin: Espaciado exterior.

4.2 <Rectangle>

¿Qué es?

Control para dibujar un rectángulo o un cuadrado.

¿Para qué se usa?

Crear separadores, bloques visuales, botones personalizados.

¿Dónde es mejor usarlo?

Formularios, fondos decorativos, diagramas.

¿Cuándo evitarlo?

Evitar si necesitas contenido complejo dentro (usar Border o Grid).

Ejemplo básico en XAML

```
<Rectangle
  Width="200"
  Height="100"
  Fill="LightGray"
  Stroke="DarkSlateBlue"
  StrokeThickness="2"
  RadiusX="10"
  RadiusY="10"
  Margin="10"/>
```

Ejemplo avanzado en XAML

```
<Grid Margin="20">
  <Rectangle Width="300" Height="150" Stroke="DarkSlateBlue" StrokeThickness="4">
    <Rectangle.Fill>
      <LinearGradientBrush StartPoint="0,0" EndPoint="1,1">
        <GradientStop Color="Lavender" Offset="0.0"/>    <GradientStop Color="SlateBlue"
Offset="1.0"/>
      </LinearGradientBrush>
    </Rectangle.Fill>
  </Rectangle>
</Grid>
```

Atributos comunes

- Fill: Relleno del rectángulo.
- Stroke: Color del borde.
- StrokeThickness: Grosor del borde.
- RadiusX, RadiusY: Radio de curvatura de las esquinas.
- Width, Height: Dimensiones.
- Margin: Espaciado exterior.

4.3 <Line>

¿Qué es?

Control para dibujar una línea recta entre dos puntos.

¿Para qué se usa?

Conectar elementos visualmente, decorar con líneas.

¿Dónde es mejor usarlo?

Diagrama de flujos, separadores de contenido, gráficos personalizados.

¿Cuándo evitarlo?

Evitar para formas complejas (usar Path).

Ejemplo básico en XAML

```
<Line
  X1="0"
  Y1="0"
  X2="200"
  Y2="0"
  Stroke="SlateBlue"
  StrokeThickness="3"
  Margin="10"/>
```

Ejemplo avanzado en XAML

```
<Canvas Width="300" Height="100" Background="White" Margin="20">
  <Line X1="10" Y1="10" X2="290" Y2="90" Stroke="SlateBlue" StrokeThickness="4"/>
</Canvas>
```

Atributos comunes

- X1, Y1: Punto de inicio.
- X2, Y2: Punto de finalización.
- Stroke: Color de la línea.
- StrokeThickness: Grosor de la línea.
- Margin: Espaciado exterior.

4.4 <Path>

¿Qué es?

Control para dibujar figuras geométricas complejas mediante rutas de vectores.

¿Para qué se usa?

Dibujar iconos personalizados, gráficos de formas libres.

¿Dónde es mejor usarlo?

Aplicaciones gráficas, interfaces decorativas, iconografía.

¿Cuándo evitarlo?

Evitar si basta con formas básicas (usar Rectangle, Ellipse, Line).

Ejemplo básico en XAML

```
<Path
  Data="M 10,100 C 100,0 200,200 300,100"
  Stroke="DarkSlateBlue"
  StrokeThickness="3"
  Fill="LightBlue"
  Margin="10"/>
```

Ejemplo avanzado en XAML

```
<Grid Margin="20">
  <Path Stroke="SlateBlue" StrokeThickness="4" Fill="Lavender">
    <Path.Data>
      <GeometryGroup>
        <EllipseGeometry Center="50,50" RadiusX="40" RadiusY="20"/>
        <RectangleGeometry Rect="90,30,60,40"/>
      </GeometryGroup>
    </Path.Data>
  </Path>
</Grid>
```

Atributos comunes

- Data: Geometría que define la forma.
- Stroke: Color del borde.
- Fill: Color de relleno interno.
- StrokeThickness: Grosor de la línea.

- Margin: Espaciado exterior.

4.5 <Transformaciones> (RotateTransform, ScaleTransform, TranslateTransform, SkewTransform, TransformGroup)

¿Qué es?

Conjunto de transformaciones aplicables a cualquier control para modificar su posición, tamaño, rotación o inclinación.

¿Para qué se usa?

Animar, mover, escalar o girar elementos visualmente.

¿Dónde es mejor usarlo?

Interfaz animada, diseños dinámicos, efectos gráficos.

¿Cuándo evitarlo?

Evitar transformar excesivamente muchos elementos en UIs muy pesadas (impacta rendimiento).

Ejemplo básico en XAML

```
<Button Content="Girar"
    Width="120" Height="40"
    Background="LightBlue"
    Foreground="DarkSlateBlue"
    Margin="10">
    <Button.RenderTransform>
        <RotateTransform Angle="45"/>
    </Button.RenderTransform>
</Button>
```

Ejemplo avanzado en XAML

```
<Grid Margin="20">
    <Rectangle Width="100" Height="100" Fill="SlateBlue">
        <Rectangle.RenderTransform>
            <TransformGroup>
                <RotateTransform Angle="30"/>
                <TranslateTransform X="50" Y="20"/>
                <ScaleTransform ScaleX="1.5" ScaleY="1.5"/>
            </TransformGroup>
        </Rectangle.RenderTransform>
    </Rectangle>
</Grid>
```

```
</Rectangle>
</Grid>
```

Atributos comunes

- Angle: Ángulo de rotación (RotateTransform).
- ScaleX, ScaleY: Escala en X e Y (ScaleTransform).
- X, Y: Desplazamiento horizontal y vertical (TranslateTransform).
- SkewX, SkewY: Inclinaciones (SkewTransform).
- Children: Colección de transformaciones en TransformGroup.

4.6 <Border>

¿Qué es?

Border es un contenedor sencillo que permite agregar un borde, un fondo y esquinas redondeadas alrededor de otro control o contenido.

¿Para qué se usa?

Envolver visualmente otros elementos para destacarlos o separarlos del resto de la interfaz.

¿Dónde es mejor usarlo?

Formularios, paneles de información, botones personalizados, separaciones visuales claras.

¿Cuándo evitarlo?

Evitar si no se necesita decoración extra o si el contenedor exterior ya tiene suficiente contraste visual.

Ejemplo básico en XAML

```
<Border
    Width="300"
    Height="150"
    Background="Lavender"
    BorderBrush="SlateBlue"
    BorderThickness="3"
    CornerRadius="15"
    Margin="10"
    Padding="10">
    <TextBlock Text="Contenido enmarcado elegantemente"
        Foreground="DarkSlateBlue"
        FontSize="16"
        HorizontalAlignment="Center"
        VerticalAlignment="Center"/>
```

</Border>

Ejemplo avanzado en XAML

```
<Grid Margin="20">
  <Border Width="300" Height="150" BorderBrush="DarkSlateBlue" BorderThickness="4"
    CornerRadius="20">
    <Border.Background>
      <LinearGradientBrush StartPoint="0,0" EndPoint="1,1">
        <GradientStop Color="Lavender" Offset="0.0"/>    <GradientStop Color="SlateBlue"
          Offset="1.0"/>
      </LinearGradientBrush>
    </Border.Background>
    <StackPanel HorizontalAlignment="Center" VerticalAlignment="Center">
      <TextBlock Text="Bienvenido" FontSize="18" FontWeight="Bold" Foreground="White"
        Margin="0,0,0,10"/>
      <Button Content="Entrar" Width="100" Background="White"
        Foreground="SlateBlue"/>
    </StackPanel>
  </Border>
</Grid>
```

Ejemplo en code-behind

// No requiere Code-Behind, todo es declarativo en este caso.

Atributos comunes

- Background: Color o brush de fondo del Border.
- BorderBrush: Color o brush del borde.
- BorderThickness: Grosor del borde.
- CornerRadius: Radio de redondeo de las esquinas.
- Child: Contenido interno (un único elemento).
- Padding: Espaciado interior entre el borde y su contenido.
- Margin: Espaciado exterior respecto a otros elementos.

4.7 <Style>

¿Qué es?

Un Style permite agrupar y aplicar un conjunto de propiedades visuales a varios controles de manera consistente.

¿Para qué se usa?

Centralizar y unificar la apariencia de controles para lograr interfaces coherentes.

¿Dónde es mejor usarlo?

Formularios extensos, sistemas con un tema visual unificado.

¿Cuándo evitarlo?

Evitar estilos excesivamente genéricos o recargados que dificulten el mantenimiento.

Ejemplo básico en XAML

```
<Window.Resources>
  <Style TargetType="Button">
    <Setter Property="Background" Value="LightBlue"/>
    <Setter Property="Foreground" Value="SlateBlue"/>
    <Setter Property="FontSize" Value="16"/>
  </Style>
</Window.Resources>

<Button Content="Aceptar" Width="150" Margin="10"/>
```

Ejemplo avanzado en XAML

```
<Window.Resources>
  <Style x:Key="BotonPersonalizado" TargetType="Button">
    <Setter Property="Background" Value="Lavender"/>
    <Setter Property="Foreground" Value="DarkSlateBlue"/>
    <Setter Property="FontSize" Value="16"/>
    <Setter Property="BorderBrush" Value="SlateBlue"/>
    <Setter Property="BorderThickness" Value="2"/>
    <Setter Property="Padding" Value="10"/>
  </Style>
</Window.Resources>

<Button Content="Guardar" Style="{StaticResource BotonPersonalizado}" Width="150"
Margin="10"/>
```

Atributos comunes

- TargetType: Tipo de control al que se aplica el estilo.
- Setters: Propiedades definidas en el estilo.
- x:Key: Identificador para aplicar el estilo manualmente.
- BasedOn: Basarse en otro estilo como plantilla.

4.8 <Trigger>

¿Qué es?

Permite cambiar propiedades automáticamente cuando cambia una propiedad del control.

¿Para qué se usa?

Responder visualmente a estados como pasar el ratón o recibir el foco.

¿Dónde es mejor usarlo?

Botones interactivos, validaciones visuales en formularios.

¿Cuándo evitarlo?

Evitar uso excesivo de triggers que puedan afectar el rendimiento o la claridad del XAML.

Ejemplo básico en XAML

```
<Button Content="Enviar" Width="150" Margin="10">
  <Button.Style>
    <Style TargetType="Button">
      <Setter Property="Background" Value="LightBlue"/>
      <Style.Triggers>
        <Trigger Property="IsMouseOver" Value="True">
          <Setter Property="Background" Value="SlateBlue"/>
          <Setter Property="Foreground" Value="White"/>
        </Trigger>
      </Style.Triggers>
    </Style>
  </Button.Style>
</Button>
```

Ejemplo avanzado en XAML

```
<TextBox Width="250" Margin="10">
  <TextBox.Style>
    <Style TargetType="TextBox">
      <Setter Property="Background" Value="White"/>
      <Setter Property="Foreground" Value="Black"/>
      <Style.Triggers>
        <Trigger Property="IsFocused" Value="True">
          <Setter Property="Background" Value="Lavender"/>
          <Setter Property="Foreground" Value="DarkSlateBlue"/>
        </Trigger>
      </Style.Triggers>
    </Style>
  </TextBox.Style>
</TextBox>
```

```

        </Style.Triggers>
    </Style>
</TextBox.Style>
</TextBox>

```

Atributos comunes

- Property: Propiedad a observar.
- Value: Valor que activa el Trigger.
- Setters: Cambios de propiedades cuando se activa.

4.9 <EventTrigger>

¿Qué es?

Activa una acción o animación basada en un evento, no en una propiedad de estado.

¿Para qué se usa?

Animaciones o reacciones inmediatas a eventos como clics o cargas.

¿Dónde es mejor usarlo?

Animaciones de entrada, botones animados, transiciones.

¿Cuándo evitarlo?

Evitar si un simple Trigger basado en propiedad sería suficiente.

Ejemplo básico en XAML

```

<Border Width="200" Height="100" Background="LightBlue" Margin="10">
    <Border.Triggers>
        <EventTrigger RoutedEvent="MouseEnter">
            <BeginStoryboard>
                <Storyboard>
                    <ColorAnimation
Storyboard.TargetProperty="(Border.Background).(SolidColorBrush.Color)"
To="SlateBlue" Duration="0:0:0.3"/>
                </Storyboard>
            </BeginStoryboard>
        </EventTrigger>
    </Border.Triggers>
</Border>

```

Ejemplo avanzado en XAML


```

<Button Content="Animar" Width="150" Height="50" Margin="10">
  <Button.Triggers>
    <EventTrigger RoutedEvent="Button.Click">
      <BeginStoryboard>
        <Storyboard>
          <DoubleAnimation Storyboard.TargetProperty="Width" To="250"
Duration="0:0:0.5"/>
        </Storyboard>
      </BeginStoryboard>
    </EventTrigger>
  </Button.Triggers>
</Button>

```

Atributos comunes

- RoutedEvent: Evento que dispara la acción.
- Actions: Lista de acciones que ejecuta el trigger (normalmente BeginStoryboard).
- Storyboard: Grupo de animaciones.

4.10 <MultiTrigger>

¿Qué es?

Permite cambiar propiedades cuando varias condiciones de propiedad se cumplen simultáneamente.

¿Para qué se usa?

Aplicar cambios visuales más sofisticados basados en múltiples estados.

¿Dónde es mejor usarlo?

Botones avanzados, formularios con validaciones combinadas.

¿Cuándo evitarlo?

Evitar combinaciones demasiado complejas que hagan el XAML ilegible.

Ejemplo básico en XAML

```

<Button Content="Enviar" Width="150" Margin="10">
  <Button.Style>
    <Style TargetType="Button">
      <Setter Property="Background" Value="LightBlue"/>
      <Style.Triggers>
        <MultiTrigger>
          <MultiTrigger.Conditions>
            <Condition Property="IsMouseOver" Value="True"/>

```

```

        <Condition Property="IsPressed" Value="True"/>
    </MultiTrigger.Conditions>
    <Setter Property="Background" Value="DarkSlateBlue"/>
    <Setter Property="Foreground" Value="White"/>
</MultiTrigger>
</Style.Triggers>
</Style>
</Button.Style>
</Button>

```

Ejemplo avanzado en XAML

```

<TextBox Width="250" Margin="10">
    <TextBox.Style>
        <Style TargetType="TextBox">
            <Setter Property="Background" Value="White"/>    <Style.Triggers>
                <MultiTrigger>
                    <MultiTrigger.Conditions>
                        <Condition Property="IsFocused" Value="True"/>
                        <Condition Property="IsEnabled" Value="True"/>
                    </MultiTrigger.Conditions>
                    <Setter Property="Background" Value="Lavender"/>
                </MultiTrigger>
            </Style.Triggers>
        </Style>
    </TextBox.Style>
</TextBox>

```

Atributos comunes

- Conditions: Colección de condiciones a comprobar.
- Setters: Cambios de propiedades si todas las condiciones son verdaderas.

4.11 <DataTrigger>

¿Qué es?

Permite aplicar cambios visuales basados en los valores de propiedades de datos (Binding).

¿Para qué se usa?

Cambiar el aspecto de un control en función de los datos a los que está ligado.

¿Dónde es mejor usarlo?

Tablas, formularios dinámicos, dashboards de datos.

¿Cuándo evitarlo?

Evitar lógicas de negocio pesadas directamente en XAML.

Ejemplo básico en XAML

```
<TextBlock Text="Estado" Width="150" Margin="10">
  <TextBlock.Style>
    <Style TargetType="TextBlock">
      <Style.Triggers>
        <DataTrigger Binding="{Binding IsActive}" Value="True">
          <Setter Property="Foreground" Value="Green"/>
        </DataTrigger>
      </Style.Triggers>
    </Style>
  </TextBlock.Style>
</TextBlock>
```

Ejemplo avanzado en XAML

```
<Border Width="200" Height="100" Margin="10">
  <Border.Style>
    <Style TargetType="Border">
      <Style.Triggers>
        <DataTrigger Binding="{Binding Estado}" Value="Error">
          <Setter Property="Background" Value="Red"/>
        </DataTrigger>
      </Style.Triggers>
    </Style>
  </Border.Style>
</Border>
```

Atributos comunes

- Binding: Fuente de datos a observar.
- Value: Valor que activa el DataTrigger.
- Setters: Propiedades que se modifican al cumplirse la condición.

4.12 <Storyboard>

¿Qué es?

Un Storyboard permite animar una o varias propiedades de un control de manera sincronizada.

¿Para qué se usa?

Crear animaciones como movimientos, cambios de color, escalados o rotaciones.

¿Dónde es mejor usarlo?

Interfaz visual dinámica, efectos de entrada y salida, atención del usuario.

¿Cuándo evitarlo?

Evitar animaciones excesivas que distraigan o saturen la UI.

Ejemplo básico en XAML

```
<Window x:Class="WpfApp11.MainWindow"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  Title="Animación en XAML" Height="200" Width="300">

  <Window.Resources>
    <Storyboard x:Key="AnimacionAncho">
      <DoubleAnimation
        Storyboard.TargetName="miBoton"
        Storyboard.TargetProperty="Width"
        To="300"
        Duration="0:0:1"/>
    </Storyboard>
  </Window.Resources>

  <Grid>
    <Button x:Name="miBoton"
      Content="Haz clic"
      Width="150"
      Height="50"
      Margin="20">
      <Button.Triggers>
        <EventTrigger RoutedEvent="Button.Click">
          <BeginStoryboard Storyboard="{StaticResource AnimacionAncho}" />
        </EventTrigger>
      </Button.Triggers>
    </Button>
  </Grid>
</Window>
```

```
</Grid>
</Window>
```

Ejemplo avanzado en XAML

```
<Button Name="btnAnimado" Content="Click para animar" Width="150" Height="50"
Margin="10" Click="btnAnimado_Click"/>
```

Ejemplo en code-behind

```
private void btnAnimado_Click(object sender, RoutedEventArgs e)
{
    Storyboard sb = new Storyboard();
    DoubleAnimation anim = new DoubleAnimation(150, 300, new
Duration(TimeSpan.FromSeconds(1)));
    Storyboard.SetTarget(anim, btnAnimado);
    Storyboard.SetTargetProperty(anim, new PropertyPath(Button.WidthProperty));
    sb.Children.Add(anim); sb.Begin();
}
```

Atributos comunes

- TargetName: Nombre del elemento a animar.
- TargetProperty: Propiedad que se anima.
- Duration: Duración de la animación.

4.13 <VisualBrush>

¿Qué es?

Permite usar cualquier control visual como un patrón para pintar otro elemento.

¿Para qué se usa?

Crear fondos dinámicos, reflejos, miniaturas de controles.

¿Dónde es mejor usarlo?

Dashboards, paneles de vista previa, decoraciones visuales.

¿Cuándo evitarlo?

Evitar en casos donde un fondo estático sería suficiente.

Ejemplo básico en XAML

```

<Window x:Class="VisualBrush.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="VisualBrush Simple" Height="200" Width="350">
    <StackPanel Margin="20">
        <!-- Botón visible que actuará como origen -->
        <Button x:Name="btnOrigen" Content="Botón Origen" Width="150" Height="50"/>
        <!-- Border que usa el botón como fondo visual -->
        <Border Width="200" Height="100" BorderBrush="Black" BorderThickness="2"
Margin="0,20,0,0">
            <Border.Background>
                <VisualBrush Visual="{Binding ElementName=btnOrigen}" />
            </Border.Background>
        </Border>
    </StackPanel>
</Window>

```

Atributos comunes

- Visual: Control visual que se utiliza como patrón.
- Stretch: Cómo se ajusta la imagen del visual.
- AlignmentX, AlignmentY: Alineación del contenido.

4.14 <DrawingBrush>

¿Qué es?

Permite pintar un área con dibujos vectoriales como líneas, rectángulos o geometrías complejas.

¿Para qué se usa?

Crear fondos detallados, patrones artísticos, iconografía decorativa.

¿Dónde es mejor usarlo?

Aplicaciones gráficas, dashboards, interfaces artísticas.

¿Cuándo evitarlo?

Evitar uso excesivo que complique el rendimiento.

Ejemplo básico en XAML

```

<Grid>
    <Rectangle Width="400" Height="250" Margin="10">
        <Rectangle.Fill>

```

```

<DrawingBrush>
  <DrawingBrush.Drawing>
    <GeometryDrawing Brush="LightBlue">
      <GeometryDrawing.Pen>
        <Pen Brush="SlateBlue" Thickness="2"/>
      </GeometryDrawing.Pen>
      <GeometryDrawing.Geometry>
        <EllipseGeometry Center="50,50" RadiusX="40" RadiusY="20"/>
      </GeometryDrawing.Geometry>
    </GeometryDrawing>
  </DrawingBrush.Drawing>
</DrawingBrush>
</Rectangle.Fill>
</Rectangle>
</Grid>

```

Atributos comunes

- Drawing: El contenido vectorial.
- TileMode: Cómo se repite el dibujo.
- Viewport: Área de repetición.

4.15 <ImageBrush>

¿Qué es?

Permite usar una imagen como relleno para pintar otro control.

¿Para qué se usa?

Fondos de botones, paneles decorativos, fondos personalizados.

¿Dónde es mejor usarlo?

Aplicaciones visuales, interfaces enriquecidas.

¿Cuándo evitarlo?

Evitar imágenes muy pesadas que ralenticen la carga.

Ejemplo básico en XAML

```

<Rectangle Width="200" Height="100" Margin="10">
  <Rectangle.Fill>
    <ImageBrush ImageSource="Images/ejemplo.png"/>
  </Rectangle.Fill>
</Rectangle>

```

```
</Rectangle.Fill>
</Rectangle>
```

Ejemplo avanzado en XAML

```
<Border Width="250" Height="150" Margin="10" CornerRadius="10">
  <Border.Background>
    <ImageBrush ImageSource="Images/fondo.jpg" Stretch="UniformToFill" />
  </Border.Background>
</Border>
```

Atributos comunes

- ImageSource: Ruta de la imagen.
- Stretch: Comportamiento de ajuste de la imagen.
- TileMode: Modo de repetición.

4.16 <DropShadowEffect>

¿Qué es?

Agrega una sombra paralela a cualquier elemento visual.

¿Para qué se usa?

Resaltar elementos importantes, dar sensación de profundidad.

¿Dónde es mejor usarlo?

Botones, paneles de notificación, títulos destacados.

¿Cuándo evitarlo?

Evitar uso abusivo en todos los elementos (recarga visual).

Ejemplo básico en XAML

```
<Button Content="Con sombra" Width="150" Margin="10">
  <Button.Effect>
    <DropShadowEffect Color="SlateBlue" BlurRadius="10" ShadowDepth="5" />
  </Button.Effect>
</Button>
```


Ejemplo avanzado en XAML

```
<Border Width="200" Height="100" Margin="10" Background="Lavender">
<Border.Effect>
  <DropShadowEffect Color="DarkSlateBlue" BlurRadius="15" ShadowDepth="8"
Direction="320"/>
</Border.Effect>
</Border>
```

Atributos comunes

- Color: Color de la sombra.
- BlurRadius: Difuminado de la sombra.
- ShadowDepth: Distancia del desplazamiento.
- Direction: Dirección de la sombra.

4.17 <BlurEffect>

¿Qué es?

Aplica un desenfoque a un control visual o imagen.

¿Para qué se usa?

Crear efectos de desenfoque para fondos, contenidos secundarios.

¿Dónde es mejor usarlo?

Paneles detrás de popups, ventanas modales, fondos atenuados.

¿Cuándo evitarlo?

Evitar desenfoques excesivos que reduzcan legibilidad.

Ejemplo básico en XAML

```
<Image Source="Images/fondo.jpg" Width="300" Height="200" Margin="10">
<Image.Effect>
  <BlurEffect Radius="5"/>
</Image.Effect>
</Image>
```

Ejemplo avanzado en XAML

```
<Grid>
<Image Source="Images/fondo.jpg" Width="300" Height="200" Margin="10">
```

```
<Image.Effect>
  <BlurEffect Radius="10"/>
</Image.Effect>
</Image>
<TextBlock Text="Título Importante" FontSize="24" Foreground="White"
HorizontalAlignment="Center" VerticalAlignment="Center"/>
</Grid>
```

Atributos comunes

- Radius: Cantidad de desenfoque aplicada.

5. CONCEPTOS ADICIONALES BREVES

Son elementos claves con los que nos tenemos que familiarizar a la hora de hacer proyectos en WPF.

5.1 <x:Name>

¿Qué es?

Permite asignar un nombre único a un control o elemento dentro de la interfaz gráfica para poder referenciarlo desde el código.

¿Para qué se usa?

- Para identificar controles en el code-behind o en otros elementos XAML.
- Permite manipular sus propiedades desde el código, como cambiar texto, visibilidad, tamaño, etc.
- Importante:
- x:Name es obligatorio si quieres acceder a un control desde el código.
- No puede haber dos controles con el mismo nombre en el mismo ámbito.

¿Dónde es mejor usarlo?

Cuando necesitas cambiar o leer propiedades del control desde el code-behind (por ejemplo, actualizar un texto, validar entradas, mostrar/ocultar elementos).

¿Cuándo evitarlo?

Cuando el control es puramente visual y no será manipulado (como decoraciones, fondos, bordes estáticos, etc.).

Ejemplo básico en XAML

```
<StackPanel Margin="10">
    <TextBlock x:Name="txtSaludo" Text="¡Hola!" FontSize="20" Margin="5"/>
    <Button Content="Cambiar Saludo" Click="BtnCambiarSaludo_Click" Width="150"
Margin="5"/>
</StackPanel>
```

Ejemplo en code-behind básico private void BtnCambiarSaludo_Click(object sender, RoutedEventArgs e)

```
{
    txtSaludo.Text = "¡Bienvenido!";
}
```

Ejemplo avanzado

¿Qué vamos a hacer?

Crearemos un pequeño formulario donde el usuario escribirá su nombre en un TextBox. Al pulsar un botón, se mostrará un saludo personalizado utilizando ese nombre.

XAML

```
<StackPanel Margin="10">
    <TextBox x:Name="txtNombre" Width="200" Height="30" Margin="5" />
    <Button Content="Saludar" Click="BtnSaludar_Click" Width="150" Margin="5"/>
    <TextBlock x:Name="txtMensaje" FontSize="18" Margin="5"/>
</StackPanel>
```

Code-behind private void BtnSaludar_Click(object sender, RoutedEventArgs e)

```
{
    if (!string.IsNullOrEmpty(txtNombre.Text))
    {
        txtMensaje.Text = $"Hola, {txtNombre.Text}!";
    } else
    {
        txtMensaje.Text = "Por favor, escribe tu nombre.";
    }
}
```

Atributos comunes

- x:Name: Identificador único para el control.
- Width / Height: Tamaño del control.
- Margin: Separación alrededor del control.
- Text / Content: Contenido que muestra el control.

5.2 <Visibility>

¿Qué es?

Visibility es una propiedad que determina si un elemento de la interfaz gráfica es visible, oculto o no participa en el diseño (no ocupa espacio).

¿Para qué se usa?

- Para mostrar u ocultar controles dinámicamente desde XAML o code-behind.
- Permite controlar la experiencia del usuario al ocultar elementos según el contexto.
- Importante:
- Tiene tres valores posibles: Visible, Hidden y Collapsed.
- Hidden oculta el elemento pero sigue ocupando espacio.
- Collapsed lo oculta y además libera su espacio en el layout.

¿Dónde es mejor usarlo?

Cuando se necesita ocultar o mostrar elementos en tiempo de ejecución según alguna condición, estado de la aplicación o interacción del usuario.

¿Cuándo evitarlo?

Cuando el control nunca va a cambiar su visibilidad o cuando se pueden usar triggers o bindings más declarativos para evitar manipulación directa desde el código.

Ejemplo básico

XAML:

```
<StackPanel Margin="10">
    <TextBlock x:Name="txtMensaje" Text="¡Hola!" FontSize="18" Visibility="Collapsed"
Margin="5"/>
    <Button Content="Mostrar Mensaje" Click="BtnMostrar_Click" Width="150"
Margin="5"/>
</StackPanel>
```

```
Code-behind: private void BtnMostrar_Click(object sender, RoutedEventArgs e)
{
    txtMensaje.Visibility = Visibility.Visible;
}
```

Ejemplo avanzado (opcional)

¿Qué vamos a hacer?

Crearemos un formulario con un CheckBox que permite mostrar u ocultar un panel de configuración adicional.

XAML:

```
<StackPanel Margin="10">
    <CheckBox x:Name="chkOpciones" Content="Mostrar opciones avanzadas"
Checked="ChkOpciones_Checked" Unchecked="ChkOpciones_Unchecked" Margin="5"/>
    <StackPanel x:Name="panelOpciones" Visibility="Collapsed" Margin="5">
        <TextBlock Text="Opción 1" />
        <TextBlock Text="Opción 2" />
    </StackPanel>
</StackPanel>
```

```
</StackPanel>
</StackPanel>
```

```
Code-behind: private void ChkOpciones_Checked(object sender, RoutedEventArgs e)
{
    panelOpciones.Visibility = Visibility.Visible;
}
```

```
private void ChkOpciones_Unchecked(object sender, RoutedEventArgs e)
{
    panelOpciones.Visibility = Visibility.Collapsed;
}
```

Atributos comunes

- Visibility: Visible, Hidden, Collapsed
- IsEnabled: Habilita o deshabilita controles (a menudo usado junto con Visibility) -
- Margin / Padding: Afectan cómo el control ocupa espacio cuando es Visible

5.3 <ItemsSource>

¿Qué es?

ItemsSource es una propiedad que permite enlazar una colección de datos a controles que muestran listas, como ListBox, ComboBox o DataGrid.

¿Para qué se usa?

- Para conectar una lista de elementos (colecciones, arrays, listas, etc.) a un control de interfaz visual.
- Permite que el contenido de controles como ListBox, ComboBox o DataGrid se genere dinámicamente a partir de datos.
- Importante:
- Requiere que el control soporte múltiples elementos (ItemsControl).
- Funciona perfectamente con Binding y MVVM.

¿Dónde es mejor usarlo?

Cuando necesitas mostrar una lista de elementos que puede cambiar en tiempo de ejecución o venir de una fuente externa (base de datos, archivo, lógica interna).

¿Cuándo evitarlo?

Cuando solo necesitas un par de elementos estáticos. En ese caso puedes definir los ítems directamente en XAML sin usar ItemsSource.

Ejemplo básico

XAML:

```
<ComboBox x:Name="comboFrutas" Width="150" Margin="10" SelectedIndex="0" />
```

Code-behind:

```
public MainWindow()
{
    InitializeComponent();
    comboFrutas.ItemsSource = new List<string> { "Manzana", "Pera", "Banana", "Fresa" };
}
```

Ejemplo avanzado (opcional)

¿Qué vamos a hacer?

Mostraremos una lista de objetos personalizados en un ListBox. Cada objeto tendrá una propiedad 'Nombre' que se mostrará.

XAML:

```
<ListBox x:Name="listProductos" DisplayMemberPath="Nombre" Width="200"
Height="100" Margin="10"/>
```

Code-behind:

```
public class Producto
{
    public string Nombre { get; set; }
}

public MainWindow()
{
    InitializeComponent();
    var productos = new List<Producto>
    {
        new Producto { Nombre = "Teclado" },    new Producto { Nombre = "Ratón" },    new
Producto { Nombre = "Monitor" }
    };
    listProductos.ItemsSource = productos;
}
```

Atributos comunes

- ItemsSource: Fuente de datos (colección)
- DisplayMemberPath: Nombre de la propiedad a mostrar como texto en la lista
- SelectedItem / SelectedValue: Ítem o valor seleccionado
- DataContext: Contexto de datos usado en bindings (relevante en MVVM)

5.4 SelectedItem

¿Qué es?

SelectedItem es una propiedad que representa el elemento actualmente seleccionado en un control que muestra una lista de opciones (como ComboBox, ListBox o DataGrid).

¿Para qué se usa?

- Para obtener o establecer el ítem seleccionado por el usuario en tiempo de ejecución.
- Permite reaccionar ante cambios de selección o preseleccionar un valor desde el código.
- Importante:
- El valor de SelectedItem es el objeto completo de la colección vinculada.
- Se puede usar junto con SelectedIndex o SelectedValue según el caso.

¿Dónde es mejor usarlo?

Cuando necesitas saber qué elemento eligió el usuario o cuando quieres establecer un valor por defecto en un control de lista.

¿Cuándo evitarlo?

Cuando solo necesitas el índice o un valor específico (entonces es más claro usar SelectedIndex o SelectedValue).

Ejemplo básico

XAML:

```
<ComboBox x:Name="comboColores" Width="150" Margin="10"
SelectionChanged="ComboColores_SelectionChanged"/>
<TextBlock x:Name="txtSeleccion" FontSize="16" Margin="10"/>
```

Code-behind:

```
public MainWindow()
{
    InitializeComponent();
    comboColores.ItemsSource = new List<string> { "Rojo", "Verde", "Azul" };
}

private void ComboColores_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    if (comboColores.SelectedItem != null)
    {
        txtSeleccion.Text = $"Has seleccionado: {comboColores.SelectedItem}";
    }
}
```

Ejemplo avanzado (opcional)

¿Qué vamos a hacer?

Mostraremos una lista de objetos 'Persona' en un ListBox y al seleccionar una, mostraremos su información.

XAML:

```
<ListBox x:Name="listPersonas" DisplayMemberPath="Nombre"
SelectionChanged="ListPersonas_SelectionChanged" Margin="10"/>
<TextBlock x:Name="txtDetalle" FontSize="14" Margin="10"/>
```

Code-behind:

```
public class Persona
{
    public string Nombre { get; set; }    public int Edad { get; set; }
}

public MainWindow()
{
    InitializeComponent();
    var personas = new List<Persona>
    {
        new Persona { Nombre = "Ana", Edad = 28 },    new Persona { Nombre = "Luis", Edad =
35 }
    };
    listPersonas.ItemsSource = personas;
}

private void ListPersonas_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    if (listPersonas.SelectedItem is Persona seleccionada)
    {
        txtDetalle.Text = $"{seleccionada.Nombre}, {seleccionada.Edad} años";
    }
}
```

Atributos comunes

- SelectedItem: Devuelve el objeto seleccionado actual.
- SelectedIndex: Índice del elemento seleccionado.
- SelectedValue: Valor específico si se usa SelectedValuePath.
- DisplayMemberPath: Propiedad visible del objeto (para mostrar nombres, por ejemplo).

5.5 SelectedIndex

¿Qué es?

SelectedIndex es una propiedad que indica el índice del elemento actualmente seleccionado en un control que maneja listas, como ListBox, ComboBox o TabControl.

¿Para qué se usa?

- Para obtener o establecer la posición (índice) del elemento seleccionado.
- Útil para seleccionar elementos por su posición o reaccionar ante cambios.
- Importante:
- El índice empieza desde 0.
- Si no hay selección, el valor es -1.

¿Dónde es mejor usarlo?

Cuando necesitas trabajar con la posición numérica de los elementos, por ejemplo, para acceder a listas en paralelo o establecer selecciones por lógica.

¿Cuándo evitarlo?

Cuando necesitas trabajar directamente con los objetos seleccionados. En esos casos es más claro usar SelectedItem o SelectedValue.

Ejemplo básico

XAML:

```
<ComboBox x:Name="comboDias" Width="150" Margin="10"
SelectionChanged="ComboDias_SelectionChanged"/>
<TextBlock x:Name="txtIndice" FontSize="16" Margin="10"/>
```

Code-behind:

```
public MainWindow()
{
    InitializeComponent();
    comboDias.ItemsSource = new List<string> { "Lunes", "Martes", "Miércoles" };
}

private void ComboDias_SelectionChanged(object sender, SelectionChangedEventArgs e)
{
    txtIndice.Text = $"Índice seleccionado: {comboDias.SelectedIndex}";
}
```

Ejemplo avanzado (opcional)

¿Qué vamos a hacer?

Asignaremos por código un elemento seleccionado por su índice al iniciar la ventana.

XAML:

```
<ListBox x:Name="listOpciones" Width="200" Height="100" Margin="10"/>
<TextBlock x:Name="txtSeleccionado" FontSize="14" Margin="10"/>
```

Code-behind:

```
public MainWindow()
{
    InitializeComponent();
    listOpciones.ItemsSource = new List<string> { "Uno", "Dos", "Tres" };
    listOpciones.SelectedIndex = 1; // Selecciona "Dos"    txtSeleccionado.Text = $"Elemento
actual: {listOpciones.SelectedItem}";
}
```

Atributos comunes

- SelectedIndex: Índice seleccionado (empieza en 0).
- SelectedItem: El objeto seleccionado (si necesitas más que la posición).
- SelectedValue: Valor si se usa SelectedValuePath. - ItemsSource: Lista de datos asociada al control.

5.6 DataContext

¿Qué es?

DataContext es una propiedad fundamental en WPF que define el origen de datos para los enlaces (bindings) dentro de un control y sus hijos.

¿Para qué se usa?

- Para establecer el objeto o contexto del cual se obtendrán los datos mediante Binding.
- Permite que los controles muestren o interactúen con datos del ViewModel o cualquier objeto.
- Importante:
- Todo Binding busca su valor relativo al DataContext.
- Si no se establece explícitamente, puede ser heredado del contenedor padre.

¿Dónde es mejor usarlo?

En aplicaciones estructuradas (especialmente con MVVM), donde se necesita enlazar múltiples propiedades de interfaz a un objeto de datos (modelo o ViewModel).

¿Cuándo evitarlo?

En proyectos muy simples donde los datos se asignan directamente desde el code-behind o se manipulan solo con eventos.

Ejemplo básico

XAML:

```
<Window.DataContext>
  <local:Persona Nombre="Lucía" Edad="30" />
</Window.DataContext>
```

```
<StackPanel Margin="10">
  <TextBlock Text="{Binding Nombre}" FontSize="16"/>
  <TextBlock Text="{Binding Edad}" FontSize="16"/>
</StackPanel>
```

Clase en C#:

```
public class Persona
{
  public string Nombre { get; set; }   public int Edad { get; set; } }
```

Ejemplo avanzado (opcional)

¿Qué vamos a hacer?

Asociamos un ViewModel al DataContext desde el code-behind.

XAML:

```
<StackPanel>
  <TextBox Text="{Binding Nombre}" Width="150" Margin="5"/>
  <TextBox Text="{Binding Edad}" Width="150" Margin="5"/>
</StackPanel>
```

Code-behind: public partial class MainWindow : Window

```
{
  public MainWindow()
  {
    InitializeComponent();
    DataContext = new Persona { Nombre = "Mario", Edad = 25 };
  }
}
```

Clase Persona:

```
public class Persona
{
  public string Nombre { get; set; }   public int Edad { get; set; } }
```

Atributos comunes

- DataContext: Objeto de datos usado como fuente de Binding.
- Binding Path: Propiedad del DataContext a la que se vincula.
- ElementName / RelativeSource: Alternativas al uso de DataContext si se desea enlazar a otro elemento visual.

5.7 StaticResource

¿Qué es?

StaticResource es una marca especial de XAML utilizada para referenciar un recurso definido previamente, como estilos, pinceles, colores o cualquier objeto estático.

¿Para qué se usa?

- Para aplicar recursos definidos previamente de forma estática (en tiempo de carga).
- Es muy común para aplicar estilos, colores, márgenes y otros valores definidos en diccionarios de recursos.
- Importante:
- El recurso debe estar definido antes (en orden o alcance visual).
- No se actualiza automáticamente si el recurso cambia en tiempo de ejecución.

¿Dónde es mejor usarlo?

Cuando necesitas reutilizar valores visuales o lógicos que no cambiarán durante la ejecución. Ideal para estilos, colores, tamaños, etc.

¿Cuándo evitarlo?

Cuando necesitas que el recurso se actualice dinámicamente en tiempo de ejecución (en ese caso se recomienda usar DynamicResource).

Ejemplo básico

XAML:

```
<Window.Resources>
  <SolidColorBrush x:Key="ColorPrincipal" Color="LightBlue"/>
</Window.Resources>

<Grid Background="{StaticResource ColorPrincipal}">
  <TextBlock Text="Bienvenido" FontSize="20" Margin="10"/>
</Grid>
```

Ejemplo avanzado (opcional)

¿Qué vamos a hacer?

Aplicaremos un estilo a varios botones usando StaticResource para mantener consistencia visual.

XAML:

```
<Window.Resources>
  <Style x:Key="BotonEstilo" TargetType="Button">
    <Setter Property="Background" Value="LightGreen"/>
    <Setter Property="FontWeight" Value="Bold"/>
    <Setter Property="Padding" Value="10,5"/>
  </Style>
```

```
</Window.Resources>
```

```
<StackPanel Margin="10">
```

```
  <Button Content="Aceptar" Style="{StaticResource BotonEstilo}"/>
```

```
  <Button Content="Cancelar" Style="{StaticResource BotonEstilo}"/>
```

```
</StackPanel>
```

Atributos comunes

- x:Key: Nombre del recurso a reutilizar.
- StaticResource: Sintaxis para referenciar el recurso.
- TargetType: Tipo de control al que aplica el estilo (cuando es un Style).
- Value: Valor estático asignado al recurso.

5.8 DynamicResource

¿Qué es?

DynamicResource es una marca de XAML que permite referenciar un recurso cuyo valor puede cambiar durante el tiempo de ejecución.

¿Para qué se usa?

- Para enlazar recursos que podrían cambiar dinámicamente durante la vida de la aplicación.
- Permite que un cambio en el recurso se refleje automáticamente en los controles que lo usan.
- Importante:
- Útil en escenarios como cambio de tema, idioma, o estilos en caliente.
- Es evaluado en tiempo de ejecución, no al cargar la interfaz.

¿Dónde es mejor usarlo?

Cuando se espera que el valor del recurso pueda modificarse en tiempo de ejecución, como en temas dinámicos o configuraciones que cambian sin reiniciar la aplicación.

¿Cuándo evitarlo?

Cuando el recurso es fijo y no cambiará durante la ejecución. En ese caso, StaticResource es más eficiente y rápido.

Ejemplo básico

XAML:

```
<Window.Resources>
```

```
  <SolidColorBrush x:Key="ColorDinamico" Color="LightCoral"/>
```

```
</Window.Resources>
```

```
<Grid Background="{DynamicResource ColorDinamico}">
```

```
<TextBlock Text="¡Hola Mundo!" FontSize="20" Margin="10"/>
</Grid>
```

Ejemplo avanzado (opcional)

¿Qué vamos a hacer?

Definiremos recursos en App.xaml y cambiaremos el color dinámicamente desde el codebehind.

App.xaml:

```
<Application.Resources>
  <SolidColorBrush x:Key="FondoApp" Color="LightBlue"/>
</Application.Resources>
```

XAML:

```
<Grid Background="{DynamicResource FondoApp}">
  <Button Content="Cambiar color" Click="CambiarColor_Click"/>
</Grid>
```

```
Code-behind: private void CambiarColor_Click(object sender, RoutedEventArgs e)
{
    Application.Current.Resources["FondoApp"] = new SolidColorBrush(Colors.LightGreen);
}
```

Atributos comunes

- **DynamicResource:** Referencia a un recurso con posibilidad de cambio en tiempo de ejecución.
- **x:Key:** Clave del recurso a modificar.
- **Application.Current.Resources:** Colección global de recursos de la aplicación.

5.9 ElementName

¿Qué es?

ElementName es una forma de hacer binding en WPF que permite enlazar una propiedad de un control a otra propiedad de otro control en la misma interfaz, usando su x:Name como referencia.

¿Para qué se usa?

- Para enlazar propiedades entre controles, por ejemplo, sincronizar tamaños, valores, selección, visibilidad, etc.
- Permite relaciones directas entre elementos de la interfaz sin usar código.
- Importante:
- El control al que se hace referencia debe tener un x:Name definido.
- Ambos controles deben estar en el mismo árbol visual para que funcione.

¿Dónde es mejor usarlo?

Cuando necesitas sincronizar propiedades entre controles en la misma vista, como el ancho de un botón con otro, o mostrar un valor dependiendo de otro control.

¿Cuándo evitarlo?

Cuando los controles no están directamente relacionados en el layout o si estás trabajando con MVVM puro, donde preferirías usar Binding a propiedades del ViewModel.

Ejemplo básico

XAML:

```
<StackPanel Margin="10">
    <Slider x:Name="sliderValor" Minimum="0" Maximum="100" Width="200"/>
    <TextBlock Text="{Binding Value, ElementName=sliderValor}" FontSize="16"
Margin="5"/>
</StackPanel>
```

Ejemplo avanzado (opcional)

¿Qué vamos a hacer?

Sincronizaremos el ancho de dos botones para que el segundo tenga siempre el mismo ancho que el primero.

XAML:

```
<StackPanel Margin="10">
    <Button x:Name="btnOriginal" Content="Botón A" Width="150"/>
    <Button Content="Botón B" Width="{Binding Width, ElementName=btnOriginal}"/>
</StackPanel>
```

Atributos comunes

- ElementName: Nombre del control del que se desea tomar la propiedad.
- Path: Propiedad específica del control que se va a enlazar.
- Mode: Modo de binding (OneWay, TwoWay, etc., opcional). - Binding: Declaración general del enlace entre elementos.

5.10 ResourceDictionary

¿Qué es?

ResourceDictionary es una estructura en XAML que permite agrupar y organizar recursos reutilizables como estilos, pinceles, colores, plantillas y más.

¿Para qué se usa?

- Para definir recursos de forma centralizada y reutilizable.
- Separa la lógica visual de la estructura de la interfaz, similar a una hoja de estilos.
- Importante:

- Puede estar en App.xaml (global), en Window/UserControl (local) o en archivos separados.
- Se usa junto a StaticResource o DynamicResource para aplicar los recursos definidos.

¿Dónde es mejor usarlo?

Cuando se desea mantener un diseño coherente y reutilizable en múltiples ventanas o controles. Ideal para estilos y temas compartidos.

¿Cuándo evitarlo?

En proyectos muy pequeños o prototipos donde no se reutilizan estilos. También cuando un recurso es usado una sola vez.

Ejemplo básico

```
Archivo: Estilos.xaml <ResourceDictionary
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
  <Style x:Key="BotonPrincipal" TargetType="Button">
    <Setter Property="Background" Value="LightBlue"/>
    <Setter Property="Foreground" Value="White"/>
    <Setter Property="FontWeight" Value="Bold"/>
    <Setter Property="Padding" Value="10"/>
  </Style>
</ResourceDictionary>
```

Uso en una ventana:

```
<Window.Resources>
  <ResourceDictionary Source="Estilos.xaml"/>
</Window.Resources>

<Button Content="Aceptar" Style="{StaticResource BotonPrincipal}"/>
```

Ejemplo avanzado (opcional)

¿Qué vamos a hacer?

Centralizaremos estilos globales en App.xaml mediante MergedDictionaries.

App.xaml:

```
<Application.Resources>
  <ResourceDictionary>
    <ResourceDictionary.MergedDictionaries>
      <ResourceDictionary Source="Themes/Colores.xaml"/>
      <ResourceDictionary Source="Themes/Estilos.xaml"/>
    </ResourceDictionary.MergedDictionaries>
  </ResourceDictionary>
</Application.Resources>
```


Ahora todos los estilos definidos allí estarán disponibles en toda la app.

Atributos comunes

- ResourceDictionary: Contenedor de recursos reutilizables.
- x:Key: Identificador de cada recurso definido.
- Source: Ruta a otro ResourceDictionary externo.
- MergedDictionaries: Permite combinar varios diccionarios.
- StaticResource / DynamicResource: Se usan para aplicar los recursos definidos.